

Improving Semantic Uncertainty Quantification in LVLMs with Semantic Gaussian Processes

Joseph Hoche¹, Andrei Bursuc², David Brellmann³, Gilles Louppe⁴, Pavel Izmailov⁵, Angela Yao⁶, and Gianni Franchi^{1,7}

¹AMIAD, Pôle Recherche, Palaiseau ²valeo.ai ³Safran Tech ⁴University of Liège
⁵New York University ⁶National University of Singapore ⁷ENSTA Paris

Abstract. Large Vision-Language Models (LVLMs) often produce plausible but unreliable outputs, making uncertainty estimation essential. Recent work on uncertainty estimates compare multiple sampled responses in a semantic space to measure their consistency. However, such comparisons are often fragile. They are sensitive to minor phrasing variations, and may fail to distinguish semantically distinct answers, making the uncertainty estimates unreliable. To address these limitations, we propose **Semantic Gaussian Process Uncertainty (SGPU)**. SGPU is a Bayesian framework that maps semantic consistency patterns in generated answers to a predictive uncertainty. SGPU maps generated answers into a dense semantic space, computes the Gram matrix of their embeddings, and summarizes their semantic configuration via the eigenspectrum. This spectral representation is then fed into a Gaussian Process Classifier that learns to map patterns of semantic consistency to predictive uncertainty, and that can be applied in both black-box and white-box settings. Across six LLMs and LVLMs on eight datasets spanning VQA, image classification, and textual QA, SGPU consistently achieves state-of-the-art calibration and discriminative performance. We further show that SGPU transfers across models and modalities, requiring minimal tuning and limited training samples. Code and data will be fully available upon acceptance.

Keywords: Semantic Uncertainty Quantification, Large Vision Language Model

1 Introduction

Large Vision-Language Models (LVLMs) extend Large Language Models (LLMs) to multimodal inputs and excel at tasks requiring visual and language understanding [25, 67, 76, 78, 79]. Despite their success, LVLMs are prone to hallucinations, with plausible but incorrect outputs that are unfaithful to the input [31, 34, 55]. This raises the need for uncertainty quantification (UQ) to detect when generated content should not be trusted [2, 21, 29, 42].

Initially, UQ methods for LVLMs were token-based measures. They operate directly on autoregressive token probabilities (e.g., log-perplexity or predictive entropy) [3, 60]. For instance, given an image with the Eiffel Tower and the

question “*In which city can I find this monument?*”, an LVLM may generate several answers that all refer to Paris (e.g., “*The figure shows the Eiffel Tower in Paris.*” and “*The Eiffel Tower is depicted in this image.*”). Token-based scores may assign different uncertainties to these phrases due to lexical and syntactic differences, even though they are semantically equivalent [3].

Semantic uncertainty methods aim to address this by estimating uncertainty score through the comparison of multiple sampled responses in a semantic space [18]. This is typically done using embeddings derived from the hidden states of LVLMs [9, 12, 32, 48] or from external models such as Natural Language Inference (NLI) systems [18, 35, 66], instruction-tuned LLMs [18, 35, 66], or sentence encoders [1, 23]. When the model produces conflicting answers (e.g., *Paris, Las Vegas, Tokyo*), semantic uncertainty should be high. In practice, however, semantic comparisons tend to be fragile. For instance, NLI models are highly sensitive to small changes in phrasing, additional correct details, or uninformative tokens that can perturb similarity scores [4, 23]. Similarly, representations from the hidden states of LVLMs are not explicitly optimized to capture the full range of semantic properties of generated answers and require white-box access to the model. Equivalent answers may receive low similarity scores, while distinct ones may appear similar. As a consequence, the estimated semantic uncertainty can be inconsistent and highly sensitive to various design and hyperparameter choices.

To address these limitations, we introduce *Semantic Gaussian Process Uncertainty* (SGPU). SGPU is a Bayesian framework for semantic UQ in LVLMs that learn to map semantic consistency patterns in generated answers to predictive uncertainty (see Figure 1). In particular, given an input (image and/or text), we sample multiple answers from an LVLM and embed each answer in a dense semantic space using an external sentence encoder specifically trained to capture sentence semantics. We then construct a Gram matrix over these embeddings and summarize the semantic configuration of the answers by its eigenspectrum. As shown in our work, this eigenspectrum serves as a compact descriptor of the semantic consistency and encodes information about the distribution of generated answers across different meanings. Intuitively, a spectrum dominated by a single large eigenvalue corresponds to a high semantic consistency, whereas a flatter spectrum indicates dispersed, semantically diverse answers and a high semantic uncertainty. SGPU feeds this spectral representation into a *Gaussian Process Classifier* (GPC), which learns to map patterns of semantic consistency to predictive confidence. As the encoder is external, SGPU can be applied both to white-box and fully black-box LVLMs. Furthermore, GPC introduces a principled Bayesian model designed to learn to capture semantic consistency patterns within the embedding space despite the representational limitations of the sentence encoder and provides calibrated uncertainty estimates.

Contributions. This work makes the following key contributions: (1) We provide an interpretation of the eigenvalues of the Gram matrix obtained from the semantic embeddings of generated answers and show that they encode useful information about their semantic consistency. In particular, in an ideal setting,

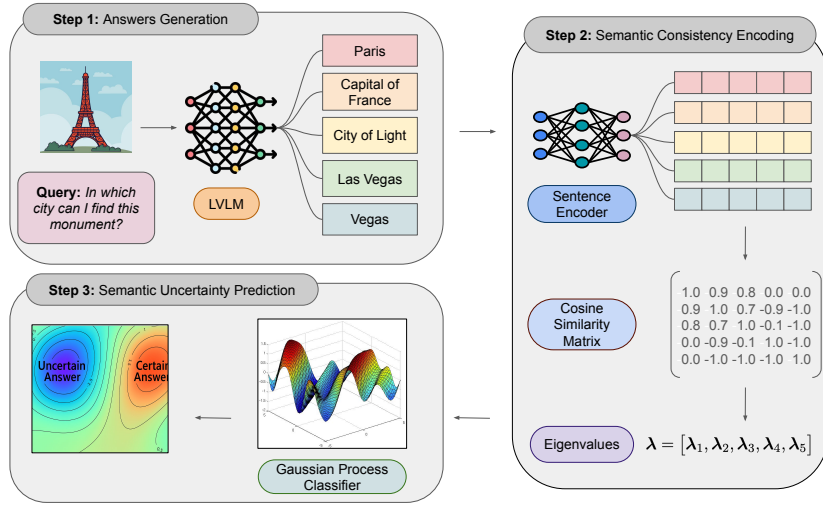


Fig. 1: Illustration of the SGPU pipeline. SGPU operates in three steps: (1) generating multiple candidate sequences for a given context (query + image), (2) encoding their semantic consistency into a single vector representation λ (Section 4.1), and (3) using this vector representation as input for a Gaussian Process Classifier to estimate the predictive semantic uncertainty / truthfulness of the generated samples (Section 4.2).

we show that the eigenspectrum depicts the distribution of generated answers across different meanings. We further analyze how representation noise affects this eigenspectrum, motivating the use of a learned predictor to better capture semantic patterns. (2) We introduce Semantic Gaussian Process Uncertainty (SGPU), a novel Bayesian framework for semantic uncertainty quantification in LVLMs that operates on the eigenspectrum of answer embeddings. To the best of our knowledge, this is the first approach to estimate LLM/LVLM confidence from multiple generated answers using a learned predictor (GPC). In our experiments, we show that SGPU consistently achieves state-of-the-art performance in both uncertainty calibration and discrimination with minimal tuning and only a small number of training samples (200 samples). (3) We observe that SGPU provides off-the-shelf, plug-and-play, and zero-shot transfer capabilities across models, datasets, and modalities, suggesting that the learned spectral representation captures general semantic uncertainty patterns that can be reused beyond the training setting.

2 Related Works

Bayesian Uncertainty Estimation. Uncertainty quantification has been a core topic in Bayesian machine learning [63], beginning with probabilistic models

such as Gaussian Processes (GPs) [82], which provide tractable posterior inference. This line of work led to Bayesian Neural Networks (BNNs) [57, 58, 65], where priors are placed over neural network weights to model uncertainty in deep architectures. Recent advances have proposed scalable approximations for BNNs [10, 21, 46], making Bayesian inference more practical for modern deep learning. However, these approaches remain computationally expensive and difficult to scale to high-dimensional vision tasks, motivating more efficient formulations tailored to computer vision [19, 59]. Alternative strategies avoid explicit posterior inference, such as deep ensembling [20, 42, 47, 81], which captures epistemic uncertainty empirically, and Laplace approximations [15, 73], which provide tractable second-order posterior estimates and have recently shown strong calibration properties in large-scale vision models [8]. Despite this progress, most Bayesian and approximate Bayesian methods focus on non-generative architectures. Extending them to large multimodal generative models such as LVLMs and LLMs remains challenging due to their scale, multimodal structure, and complex latent spaces. This limitation motivates our work.

Uncertainty Quantification in LLM and LVLM. Several recent methods have been proposed to enhance reliability by estimating uncertainty in the predictions of LLMs and LVLMs. These include self-verbalization approaches that ask the LVLM itself for its confidence [13, 14, 38, 53, 61]; methods that estimate uncertainty based on internal representations and token-level confidence from single generations [5, 9, 17, 50, 60, 64, 68, 72]; techniques that assess uncertainty through response changes under perturbations [22, 52, 84]; and methods that estimate uncertainty by measuring their consistency across multiple generated responses via their semantics using external models, internal representations and confidence scores [1, 18, 23, 36, 40, 43, 48, 54, 60, 66, 69]. Our method belongs to the last category by quantifying semantic uncertainty by measuring consistency between responses with an external sentence embedding model.

3 Preliminaries

Given an input $\mathbf{x}$ which may contain images and/or text, an LVLM parameterized by θ autoregressively generates an output response sequence of tokens $\mathbf{y} = \{\mathbf{y}_t\}_{t=1}^T$, where $\mathbf{y}_t$ denotes the t^{th} output token and T is the length of the output sequence. The probability of the generated sequence $\mathbf{y}$ is the joint probability of tokens, defined as the product of conditional token probabilities:

$$p(\mathbf{y} \mid \mathbf{x}, \theta) = \prod_{t=1}^T p(\mathbf{y}_t \mid \mathbf{x}, \mathbf{y}_{<t}, \theta), \quad \text{where } \mathbf{y}_{<t} = \{\mathbf{y}_1, \dots, \mathbf{y}_{t-1}\}. \quad (1)$$

3.1 Uncertainty Estimation

Log-Perplexity. A straightforward uncertainty measure is the average negative log-likelihood of generated tokens or the log-perplexity [60, 64]. Log-perplexity is

defined as the arithmetic mean of the log-probabilities of the generated tokens:

$$\log \text{PPL}(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta}) = -\frac{1}{T} \sum_{t=1}^T \log p(\mathbf{y}_t \mid \mathbf{x}, \mathbf{y}_{<t}, \boldsymbol{\theta}) = -\frac{1}{T} \log p(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta}). \quad (2)$$

Predictive Entropy. Another baseline is predictive entropy [18, 38, 54]. Given an input $\mathbf{x}$, predictive entropy is defined as the mean log-perplexity computed over a set of N generated candidate sequences $\mathcal{Y} := \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)}\}$:

$$H(\mathcal{Y} \mid \mathbf{x}, \boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \log \text{PPL}(\mathbf{y}^{(i)} \mid \mathbf{x}, \boldsymbol{\theta}). \quad (3)$$

$H(\mathcal{Y} \mid \mathbf{x}, \boldsymbol{\theta})$ provides a Monte-Carlo approximation of the Shannon entropy $\mathbb{E}_{\mathbf{y}}[-\log p(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta})]$ over outputs generated by the LVLm. The log-perplexity and the predictive entropy reach their highest values when the output is uncertain within the token space.

Semantic Entropy. Given an input $\mathbf{x}$, semantic entropy is defined over a set of generated sequences $\mathcal{Y} := \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)}\}$ that are partitioned into semantically consistent clusters $\mathcal{C} := \{\mathcal{C}_k\}_{k=1}^K$, such that sequences within each cluster share the same meaning. In practice, clustering is performed either by using bidirectional entailment predictions from NLI models [18, 40, 66] and instruction-tuned LLMs [18, 35, 66] or by using embeddings derived from hidden states of the LVLm [48] and external models [1]. The probability mass $p(\mathcal{C}_k \mid \mathbf{x}, \boldsymbol{\theta})$ assigned to each cluster $\mathcal{C}_k$ is defined as the sum of normalized sequence probabilities of all outputs assigned to that cluster:

$$p(\mathcal{C}_k \mid \mathbf{x}, \boldsymbol{\theta}) = \sum_{\mathbf{y} \in \mathcal{C}_k} \tilde{p}(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta}), \quad (4)$$

where $\tilde{p}(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta}) = \frac{p(\mathbf{y} \mid \mathbf{x}, \boldsymbol{\theta})}{\sum_{i=1}^N p(\mathbf{y}^{(i)} \mid \mathbf{x}, \boldsymbol{\theta})}$ for $\mathbf{y} \in \mathcal{Y}$. Using the Rao-Blackwellized Monte Carlo estimator, the semantic entropy [18, 40] is approximated as:

$$H_{\text{SE}}(\mathcal{Y} \mid \mathbf{x}, \boldsymbol{\theta}) = -\sum_{k=1}^K p(\mathcal{C}_k \mid \mathbf{x}, \boldsymbol{\theta}) \log p(\mathcal{C}_k \mid \mathbf{x}, \boldsymbol{\theta}) \quad (5)$$

and measures the dispersion of probability mass across distinct meanings. A low H_{SE} value indicates that model responses are tightly clustered around a single meaning, whereas a high value reflects greater semantic diversity in the outputs.

Discrete Semantic Entropy. Discrete semantic entropy [18, 40] is another extension of semantic entropy for black-box LVLms. It approximates the probability mass $p(\mathcal{C}_k \mid \mathbf{x}, \boldsymbol{\theta})$ assigned to each cluster $\mathcal{C}_k$ with the empirical cluster probability $|\mathcal{C}_k|/n$, yielding:

$$H_{\text{DSE}}(\mathcal{Y} \mid \mathbf{x}, \boldsymbol{\theta}) = -\sum_{k=1}^K \frac{|\mathcal{C}_k|}{n} \log \frac{|\mathcal{C}_k|}{n}. \quad (6)$$

3.2 Latent Uncertainty Estimation

Instead of assessing uncertainty using token likelihoods, alternative approaches leverage dense semantic information retained within the internal states of LVLMS to measure semantic divergence.

Semantic Volume. Given a context $\mathbf{x}$ and a set of generated candidate sequences $\mathcal{Y} := \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)}\}$, the semantic volume is defined over the embedding matrix $\Phi = [\phi_1, \phi_2, \dots, \phi_N] \in \mathbb{R}^{d \times N}$, where each $\phi_i \in \mathbb{R}^d$ depicts the sentence embedding of the generated answer $\mathbf{y}^{(i)}$ within the d -dimensional semantic space of the LVLMS. Note that the sentence embedding can be derived either by averaging the token embedding or by taking the last token embedding from a chosen layer or attention head [5, 12, 17, 32, 68]. Let $\Sigma = \Phi^T \left[\mathbf{I}_d - \frac{\mathbf{1}_d \mathbf{1}_d^T}{d} \right] \Phi$ denote the empirical covariance matrix of Φ . The semantic volume¹ is then defined as [12, 74, 85, 86]:

$$V(\mathcal{Y} \mid \mathbf{x}, \theta) = \log \det(\Sigma + \alpha \mathbf{I}_N) = \sum_{i=1}^N \log(\lambda_i), \quad (7)$$

where $\det(\mathbf{X})$ represents the determinant of matrix $\mathbf{X}$, $\alpha > 0$ is a small regularization term to make Σ full rank, and $\lambda = [\lambda_1, \dots, \lambda_N]$ denotes eigenvalues of the matrix $\Sigma + \alpha \mathbf{I}_N$. Variants instead define Σ using the cosine similarity matrix [43, 52] or the Gram matrix [75] and combine Σ with probability responses [43]. The semantic volume can be interpreted as the differential entropy within the sentence-level LVLMS embedding space [12].

4 Semantic Gaussian Process Uncertainty

In this section, we present the details of our proposed Semantic Gaussian Process Uncertainty (SGPU) framework, which is designed to quantify semantic uncertainty in both black-box and white-box LVLMS. The complete pipeline is depicted in Figure 1. We start by generating multiple candidate sequences $\mathcal{Y} := \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)}\}$ for a given context $\mathbf{x}$ (Step 1). Then, using an external sentence embedding model, we embed each answer in a dense semantic space and encode their semantic consistency into single vector representation (Step 2). Finally, this aggregated representation is passed to a Gaussian Process classifier (GPC), which learns to map patterns of semantic consistency to a probability score indicating confidence in the truthfulness of the generated samples (Step 3). Details and justifications of how the semantic consistency of generated answers is encoded are provided in Section 4.1 and the Gaussian Process Classifier is described in Section 4.2.

¹ The term ‘‘volume’’ arises from the geometric interpretation that $\det(\Phi^T \Phi)$ is the volume of the parallelepiped spanned by $\{\phi_i\}_{i=1}^n$.

4.1 Semantic Consistency Encoding

Sentence Embedding. To capture the semantic consistency of N generated answers $\mathcal{Y} := \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)}\}$, we embed each generated answer $\mathbf{y}^{(i)}$ in $\mathcal{Y}$ in a dense semantic space using an external sentence encoder $E(\cdot)$. For each generated answer $\mathbf{y}^{(i)}$ in $\mathcal{Y}$, we denote its d -dimensional normalized embedding vector as $\phi_i \in \mathbb{R}^d$, with $\|\phi_i\| = 1$. The sentence embedding can be obtained either by averaging the token embeddings, by using the embedding of the last token, or by taking the embedding of a designated special token.

This section provides an interpretation of the eigenvalues of the Gram matrix Σ of the embedding matrix Φ from the answer set $\mathcal{Y}$. The Gram matrix is

$$\Sigma = \Phi^T \Phi \in \mathbb{R}^{N \times N}, \text{ where } \Phi = [\phi_1, \dots, \phi_N] \in \mathbb{R}^{d \times N}. \quad (8)$$

Its eigenvalues encodes information about the semantic consistency among the generated answers in $\mathcal{Y}$. To show this, we make the following assumption.

Assumption 1 (Ideal Sentence Encoding) *Let $p \geq N$. For the set of generated answers $\mathcal{Y} := \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)}\}$, we assume there exists an ideal sentence encoder $E^*(\cdot)$ such that*

$$\phi_i^{*T} \phi_j^* = \begin{cases} 1 & \text{if } \mathbf{y}^{(i)} \text{ and } \mathbf{y}^{(j)} \text{ share the same meaning} \\ 0 & \text{otherwise,} \end{cases}$$

where $\phi_i^* = E^*(\mathbf{y}^{(i)}) \in \mathbb{R}^p$ depicts the p -dimensional normalized embedding vector (with $\|\phi_i^*\| = 1$) of the generated answer $\mathbf{y}^{(i)} \in \mathcal{Y}^2$. We denote by $\Phi^* = [\phi_1^*, \dots, \phi_N^*] \in \mathbb{R}^{p \times N}$ the ideal embedding matrix of $\mathcal{Y}$ and by $\Sigma^* = \Phi^{*T} \Phi^* \in \mathbb{R}^{N \times N}$ its Gram matrix.

Lemma 1 below shows that the eigenvalues of the Gram matrix $\Sigma^* = \Phi^{*T} \Phi^* \in \mathbb{R}^{N \times N}$ obtained from an ideal sentence encoder capture meaningful information about the semantic consistency among the generated answers in $\mathcal{Y}$ (see proof in Appendix A).

Lemma 1 (Ideal Semantic Consistency Vector). *Let $\lambda^* = [\lambda_1^*, \dots, \lambda_N^*]^T \in \mathbb{R}^N$ be the vector of eigenvalues of Σ^* ordered such that $\lambda_1^* \geq \dots \geq \lambda_N^* \geq 0$. Then, λ^* depicts the repartition of generated answers $\mathcal{Y}$ across meanings, where:*

1. *The sum of eigenvalues equals the number of generated answers N , i.e., $\sum_{i=1}^N \lambda_i^* = N$.*
2. *The number of nonzero eigenvalues in λ^* corresponds to the number of distinct semantic meanings present in the generated answers $\mathcal{Y}$, i.e., the number of ideal distinct semantic clusters.*
3. *Each nonzero λ_i^* is equal to the number of generated answers in $\mathcal{Y}$ that share the same semantic meaning, i.e., the size of an ideal semantic cluster.*

² Note that we can have $p \neq d$.

Remark 1. Intuitively, the vector λ^* serves as a compact yet rich descriptor of the semantic consistency within $\mathcal{Y}$. A high semantic consistency across the generated answers is characterized by an eigenvalue λ_1^* close to N , which indicates that all answers share the same meaning. In contrast, a flatter spectrum λ^* implies that generated answers are spread out across multiple distinct meanings.

Leveraging Weyl’s inequality, Lemma 2 depicts how the eigenvalues of the Gram matrix $\Sigma = \Phi^T \Phi$ (equation 8) derived from an external sentence encoder approximate the ideal semantic consistency vector λ^* (see proof in Appendix A).

Lemma 2. *Let $\lambda \in \mathbb{R}^N$ be the vector of eigenvalues of Σ ordered such that $\lambda_1 \geq \dots \geq \lambda_N \geq 0$. For all $i \in [N]$, we have*

$$\delta_i = \lambda_i - \lambda_i^* \in [-\sqrt{N(N-1)}\epsilon, \sqrt{N(N-1)}\epsilon],$$

where $\epsilon = \max_{i,j} |\phi_i^T \phi_j - \phi_i^{*T} \phi_j^*| \in [0, 2]$. Besides, $\sum_{i=1}^N \delta_i = 0$.

Remark 2. $\epsilon = \max_{i,j} |\phi_i^T \phi_j - \phi_i^{*T} \phi_j^*|$ measures the maximum semantic similarity error over the generated answers $\mathcal{Y}$ between an ideal sentence encoder $E^*(\cdot)$ and the external sentence encoder $E(\cdot)$. A low ϵ indicates that the semantic consistency vector λ depicts the repartition of generated answers $\mathcal{Y}$ across different meanings (Lemma 1).

Remark 3. Although we have $\sum_{i=1}^N \lambda_i = \sum_{i=1}^N \lambda_i^* = N$, as the semantic similarity error ϵ increases, the information captured by λ about the distribution of generated answers $\mathcal{Y}$ across different meanings is perturbed. Such perturbation may limit the reliability of direct UQ measures based on the eigenvalues of Σ , such as the semantic volume [12] or the Von Neumann entropy [32] (see Appendix B for further details on how representation noise affects the semantic consistency vector λ).

We propose to exploit the information about the repartition of generated answers $\mathcal{Y}$ across meanings encoded in the semantic consistency vector λ . To improve the robustness to semantic similarities errors, we train a classifier to learn to map semantic consistency patterns encoded in λ to predictive confidence as discussed in the following section.

4.2 Semantic Uncertainty Prediction

Following prior works [7, 18, 32, 40, 61], we frame semantic uncertainty evaluation as a binary classification task that predicts whether to rely on model generations for a given context. In particular, our approach leans on the common assumption in semantic uncertainty methods that high semantic consistency among generated responses indicates correctness, whereas significant variation indicates potential hallucinations.

Binary Classification Problem. We assume we have a training dataset $\mathcal{D} := \{\mathbf{x}^{(i)}, \mathcal{Y}^{(i)}\}_{i=1}^M$, where $\mathcal{Y}^{(i)} := \{\mathbf{y}_i^{(1)}, \dots, \mathbf{y}_i^{(N)}\}$ depicts a set of N candidate responses generated by the LVLM for the given context $\mathbf{x}^{(i)}$. To frame the problem into a binary classification task, we transform the dataset $\mathcal{D}$ into $\tilde{\mathcal{D}} := \{\boldsymbol{\lambda}^{(i)}, l^{(i)}\}_{i=1}^M$, where $\boldsymbol{\lambda}^{(i)} \in \mathbb{R}^N$ denotes the semantic consistency vector of the generated responses $\mathcal{Y}^{(i)}$ for the given context $\mathbf{x}^{(i)}$ (Lemma 2) and $l^{(i)}$ is the binary label indicating the correctness of generated answers $\mathcal{Y}^{(i)}$. Correctness of predicted answers is commonly used as a proxy for semantic uncertainty [7, 18, 32, 40, 61] since uncertain generations are less likely to be correct. Following recommendations in [32], for each generated response $\mathbf{y}_i^{(j)}$ relative to the context $\mathbf{x}^{(i)}$ in $\mathcal{Y}^{(i)}$, we evaluate the correctness with respect to a reference answer $\tilde{\mathbf{y}}_i$ using an LLM-as-judge approach. The truthfulness label $l^{(i)}$ that indicates the correctness of generated answers $\mathcal{Y}^{(i)}$ is then determined by the majority vote (see Appendix D for further details).

Gaussian Process Classifier. Given a set of generated outputs $\mathcal{Y} := \{\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(N)}\}$ related to a given context $\mathbf{x}$, the objective is to predict the truthfulness label $l \in \{0, 1\}$ of these generated samples using the semantic consistency vector $\boldsymbol{\lambda} \in \mathbb{R}^N$ defined in Lemma 2. We do this by computing the label probability

$$p(l \mid \mathcal{Y}, \mathbf{x}, \boldsymbol{\theta}) = p(l \mid \boldsymbol{\lambda}) \quad (9)$$

with a Gaussian Process Classifier (GPC) [70] trained on the dataset $\tilde{\mathcal{D}} := \{\boldsymbol{\lambda}^{(i)}, l^{(i)}\}_{i=1}^M$. In the following, we denote by $\mathbf{l} = [l^{(1)}, \dots, l^{(M)}]^T \in \mathbb{R}^M$ and $\mathbf{A} = [\boldsymbol{\lambda}^{(1)}, \dots, \boldsymbol{\lambda}^{(M)}]^T \in \mathbb{R}^{M \times N}$ the collections of labels and semantic consistency vectors, respectively. For a fixed $\boldsymbol{\lambda}$, the GPC models $p(l \mid \boldsymbol{\lambda})$ as a Bernoulli distribution with parameter $p(l = 1 \mid \boldsymbol{\lambda}) = s(f(\boldsymbol{\lambda}))$, where $f(\cdot)$ is a latent function and $s : \mathbb{R} \rightarrow [0, 1]$ is a sigmoid function. The latent function $f(\cdot)$ is assumed to follow a Gaussian Process (GP) prior

$$f \sim \mathcal{GP}(m(\boldsymbol{\lambda}), k(\boldsymbol{\lambda}, \boldsymbol{\lambda}')), \quad (10)$$

where $m(\cdot)$ is the mean function and $k(\cdot, \cdot)$ is a kernel function. This implies that the evaluation vector of $f(\cdot)$ on $\mathbf{A}$ defined as $\mathbf{f} = [f(\boldsymbol{\lambda}_1), \dots, f(\boldsymbol{\lambda}_M)]^T \in \mathbb{R}^M$ is drawn from a multivariate Gaussian distribution $\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{K})$, where $\boldsymbol{\mu} = [m(\boldsymbol{\lambda}_1), \dots, m(\boldsymbol{\lambda}_M)]^T$ and $\mathbf{K}_{ij} = k(\boldsymbol{\lambda}_i, \boldsymbol{\lambda}_j)$. Since neither of the class labels is considered more probable than the others, the prior mean is usually set to zero. The kernel function $k(\cdot, \cdot)$ encodes prior beliefs about the properties of the latent function $f(\cdot)$. The posterior distribution over the latent values $\mathbf{f}$ at the observed $\mathbf{A}$ is defined as:

$$p(\mathbf{f} \mid \tilde{\mathcal{D}}) = \frac{1}{p(\tilde{\mathcal{D}})} p(\mathbf{l} \mid \mathbf{f}) p(\mathbf{f} \mid \mathbf{A}), \quad (11)$$

where $p(\tilde{\mathcal{D}})$ denotes the marginal likelihood and $p(\mathbf{l} \mid \mathbf{f}) = \prod_{j=1}^M s(l^{(j)} f(\boldsymbol{\lambda}_j))$ the joint likelihood of the independent Bernoulli variables in $\mathbf{l}$. The distribution for the latent function value $f = f(\boldsymbol{\lambda})$ at a new semantic consistency vector $\boldsymbol{\lambda}$ is

obtained by averaging over the posterior distribution

$$p(f \mid \boldsymbol{\lambda}, \tilde{\mathcal{D}}) = \int p(f \mid \mathbf{f}, \tilde{\mathcal{D}}) p(\mathbf{f} \mid \tilde{\mathcal{D}}) d\mathbf{f} \quad (12)$$

and the predictive label probability $p(l \mid \mathcal{Y}, \mathbf{x}, \boldsymbol{\theta}, \tilde{\mathcal{D}})$ is computed as the expectation:

$$p(l \mid \mathcal{Y}, \mathbf{x}, \boldsymbol{\theta}, \mathcal{D}) = p(l \mid \boldsymbol{\lambda}, \tilde{\mathcal{D}}) = \int p(l \mid f) p(f \mid \boldsymbol{\lambda}, \tilde{\mathcal{D}}) df,$$

where $p(l \mid f) = s(lf(\boldsymbol{\lambda}))$. Note that neither the marginal likelihood, nor the posterior itself, or predictions can be computed analytically, so approximations such as Monte-Carlo or Laplace approximations are needed [70]. GPC provides the advantage of direct access to its internal uncertainty estimates (through the predictive standard deviation), requires only a small amount of training data, and is typically well-calibrated. Moreover, it has strong potential to generalize effectively to new or varying inputs. More detailed motivations can be found in Appendix E.

5 Experiments

5.1 Overall Performance Comparison

Datasets and Models. In this section, we evaluate our method across diverse tasks, datasets, and architectures. Our evaluation is conducted on visual question answering (VQA), image classification, and textual Question Answering (QA) tasks. For VQA, we consider four datasets: **ADVQA** [51], **VQARAD** [44], **OKVQA** [62], and **VizWiz** [26]. For image classification, we use **CIFAR10** [39] and **Imagenette** [30]; while for the QA task, we use **TriviaQA** [37] and **PopQA** [11]. Experiments on vision and VQA tasks are performed with Qwen2.5-VL-3B [80], Qwen2.5-VL-7B [6], llava-mistral-7b [56], idfics2-8b [45]. We conduct experiments with Llama-3.1-8B [80] for QA datasets. More details on the experimental protocol can be found in Appendix C.1.

Table 1: SGPU consistently improves AUROC and AUARC while reducing ECE. Evaluation metrics for vision datasets averaged across all LVLM architectures. **Bold** values indicates best performance, underline values indicate the second-best.

Method	CIFAR10			OKVQA			VQARAD			ADVQA			VizWiz			Imagenette		
	AUROC $\uparrow$	AUARC $\downarrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\downarrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\downarrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\downarrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\downarrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\downarrow$	ECE $\downarrow$
SE	0.823	0.832	0.075	0.783	0.792	0.232	0.691	0.621	0.282	0.673	0.658	0.149	0.785	<u>0.657</u>	0.219	0.604	0.617	0.309
DSE	0.818	0.832	0.070	0.753	0.787	0.213	0.691	0.599	0.292	0.675	0.659	0.152	0.787	0.651	0.230	0.605	0.614	0.285
PE	0.812	0.824	0.081	0.737	0.783	0.109	0.705	0.624	<u>0.132</u>	0.672	0.657	0.168	0.764	0.645	<u>0.112</u>	0.624	<u>0.641</u>	0.260
KLE-Heat	0.827	<u>0.838</u>	0.084	0.731	0.782	0.191	0.644	0.640	0.230	0.688	<u>0.667</u>	0.168	0.746	0.634	0.235	0.589	0.616	0.372
KLE-Matern	0.832	0.837	<u>0.066</u>	0.762	<u>0.794</u>	<u>0.075</u>	0.684	<u>0.634</u>	0.159	<u>0.700</u>	0.672	<u>0.105</u>	0.765	0.651	0.137	0.590	0.616	0.301
UMPIRE	0.826	0.800	0.122	<u>0.768</u>	0.777	0.236	<u>0.724</u>	0.622	0.260	0.667	0.621	0.229	<u>0.798</u>	0.616	0.190	0.609	0.600	0.338
Cos Eigenscore	<u>0.841</u>	0.770	0.127	0.750	0.738	0.123	0.702	0.618	0.201	0.635	0.587	0.201	0.777	0.588	0.156	0.625	0.604	0.336
Cov Eigenscore	0.787	0.750	0.243	0.685	0.705	0.280	0.605	0.616	0.234	0.620	0.581	0.324	0.711	0.584	0.158	<u>0.769</u>	0.576	<u>0.094</u>
SGPU (ours)	0.882	0.878	0.013	0.788	0.810	0.039	0.752	0.740	0.108	0.711	0.701	0.040	0.828	0.818	0.027	<u>0.748</u>	0.774	<u>0.121</u>

Table 2: SGPU consistently improves AUROC and AUARC while reducing ECE. Evaluation metrics for LVLM architectures averaged across all vision datasets. **Bold** values indicates best performance, underline values indicate the second-best.

Method	Qwen2.5-VL-3B			Qwen2.5-VL-7B			idefics2-8b			llava-mistral-7b		
	AUROC $\uparrow$	AUARC $\uparrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\uparrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\uparrow$	ECE $\downarrow$	AUROC $\uparrow$	AUARC $\uparrow$	ECE $\downarrow$
SE	0.731	0.683	0.165	0.713	0.690	0.244	0.698	0.720	0.250	0.750	0.691	0.186
DSE	0.730	0.681	0.154	0.712	0.687	0.243	0.672	0.716	0.243	0.750	0.678	0.189
PE	0.726	0.682	<u>0.087</u>	0.713	0.692	0.107	0.702	0.724	0.252	0.736	0.685	0.127
KLE-Heat	0.726	<u>0.684</u>	0.164	0.715	0.692	0.207	0.688	0.714	0.249	0.687	0.694	0.233
KLE-Matern	<u>0.747</u>	0.693	0.145	<u>0.724</u>	<u>0.698</u>	<u>0.097</u>	0.693	0.715	0.230	0.724	<u>0.697</u>	<u>0.089</u>
UMPIRE	0.722	0.618	0.158	0.709	0.658	0.278	0.714	0.727	0.285	<u>0.782</u>	0.688	0.195
Cos Eigenscore	0.699	0.559	0.157	0.714	0.625	0.158	0.736	<u>0.726</u>	0.266	0.737	0.694	0.180
Cov Eigenscore	0.653	0.554	0.196	0.717	0.631	0.177	0.765	0.678	<u>0.229</u>	0.709	0.679	0.288
SGPU (ours)	0.810	0.805	0.052	0.766	0.767	0.049	<u>0.746</u>	0.753	0.094	0.817	0.822	0.038

Evaluation Metrics. Following previous work [18, 32, 40, 43], we evaluate uncertainty methods using: AUROC [28], AUARC [41], and ECE [24]. AUROC (Area Under the Receiver Operating Curve) is a performance metric for binary classifiers, allowing it to assess whether an uncertainty estimation metric effectively distinguishes between correct and incorrect generations. Higher scores are better, with perfect uncertainty scoring 1 while a random uncertainty measure would score 0.5. AUARC (Area Under the Accuracy-Rejection Curve) measures robustness under selective prediction by averaging accuracy after progressively discarding uncertain samples, where higher AUARC means errors are concentrated in low-confidence regions. ECE (Expected Calibration Error) quantifies how well predicted probabilities align with empirical accuracies, where lower values indicate better calibration. More details on the metrics can be found in Appendix C.3.

Baselines. We evaluate SGPU using a Mattern kernel for the GPC and generated answers are embedded with all-MiniLM-L6-v2 [71]. We compare our proposal with popular uncertainty-based methods: Predictive Entropy (PE) [18, 40], Semantic Entropy (SE) [40], Discrete Semantic Entropy (DSE) [18], Kernel Language Entropy (KLE) with a Heat Kernel [66], KLE with a Matern Kernel [66], UMPIRE [43], Cov Eigenscore [12], and Cos Eigenscore [12], a variant of Cov Eigenscore, but based on cosine similarity matrix instead (see 3.1). More details can be found in Appendix C.2.

Results and Discussions. Tables 1 & 2 present an overall comparison of uncertainty estimation methods across several datasets and LVLM architectures. Results show that SGPU achieves higher AUROC and AUARC scores while maintaining a lower calibration error (ECE) compared to the baselines. Due to the inherently open-ended nature of free-form NLG, estimating uncertainty is more challenging on VQA tasks than for classification tasks. While baselines methods provide partial improvements in AUROC and AUARC, SGPU significantly outperforms them on these evaluation metrics and maintains a low calibration error (ECE). In particular, SGPU performs significantly better on both Qwen2.5-VL-3B and llava-mistral-7b, where it improves both discrimination and calibration measures. On the medical dataset **VQARAD**, baseline methods show significant degradations, whereas SGPU successfully maintains

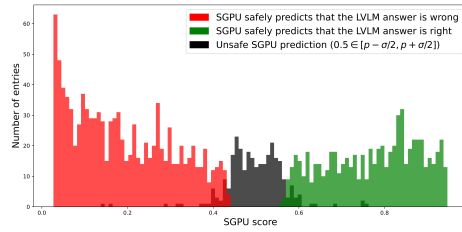


Fig. 2: Detection of Unsafe SGPU Predictions with `llava-mistral-7b` on **VIZWIZ**. Unsafe SGPU predictions are defined when $0.5 \in [p - \sigma/2, p + \sigma/2]$, where p is the SGPU score and σ is its associated predicted standard deviation.

a better AUROC and a lower ECE. Furthermore, SGPU remains robust and achieves best performance on tasks with adversarial inputs (dataset **ADVQA**). More detailed experiments (LVLM and LLM) can be found in Appendix F.

Detection of Unsafe SGPU Prediction. As shown in Figure 2, one motivation for using a GPC is its ability to provide a principled estimate of the predictive variance for each output. This variance depicts the confidence of SGPU and not the LVLM internal uncertainty. In practice, this allows us to detect when SGPU is not confident enough to make a reliable judgment (unsafe prediction); further details in Appendix E.

5.2 Transfer Experiments and Generalization

Cross-Model Transferability. An interesting aspect is that the trained SGPU generalizes well across different LVLM architectures in plug-and-play manner without any adjustment or fine-tuning. Transfer experiments in Table 3 show that although performance slightly decreases when SGPU is trained on one LVLM and evaluated on another, its predictions remain meaningful. For instance, SGPU trained on **Qwen2.5-VL-3B** and evaluated on **Qwen2.5-VL-7B** achieves the same AUROC performance results with only a moderate drop in AUARC. As shown in Appendix C.5, the model transferability remains especially strong on **CI-FAR10**, where visual patterns are simpler and more consistent. Such a result confirms that the semantic consistency vector λ is not entirely tied to a single LVLM architecture and captures stable patterns of semantic consistency across responses. Furthermore, SGPU can be reused across models without requiring full retraining. This suggests that it may serve as a general uncertainty prior for LVLMs, enabling more flexible and efficient adaptation in new domains and across different model families.

Cross-Modality Transferability. Transfer experiments in Table 4 show that although performance slightly decreases when SGPU is trained on one modality and evaluated on another, its predictions remain meaningful. For instance,

Table 3: SGPU generalizes well across different LVLM architectures. Each row shows the SGPU trained on the outputs of one LVLM and tested on another for the **VQARAD** dataset. **Bold** values indicates best performance, underline values indicate the second-best.

Trained GP	Tested on	AUROC↑	AUARC↑	ECE↓
Qwen2.5-VL-7B	Qwen2.5-VL-7B	0.730	0.710	0.060
Qwen2.5-VL-3B	Qwen2.5-VL-7B	<u>0.730</u>	<u>0.687</u>	<u>0.082</u>
llava-mistral-7b	Qwen2.5-VL-7B	0.729	0.657	0.125
idefics2-8b	Qwen2.5-VL-7B	0.724	0.655	0.068
Qwen2.5-VL-3B	Qwen2.5-VL-3B	0.770	0.770	0.060
Qwen2.5-VL-7B	Qwen2.5-VL-3B	0.700	<u>0.662</u>	<u>0.081</u>
llava-mistral-7b	Qwen2.5-VL-3B	<u>0.727</u>	0.632	0.135
idefics2-8b	Qwen2.5-VL-3B	0.684	0.622	0.064
idefics2-8b	idefics2-8b	0.750	0.664	0.278
Qwen2.5-VL-3B	idefics2-8b	<u>0.636</u>	0.588	0.191
Qwen2.5-VL-7B	idefics2-8b	0.526	0.507	<u>0.086</u>
llava-mistral-7b	idefics2-8b	0.518	<u>0.627</u>	0.221
llava-mistral-7b	llava-mistral-7b	0.758	0.815	0.045
Qwen2.5-VL-3B	llava-mistral-7b	<u>0.754</u>	0.698	<u>0.199</u>
Qwen2.5-VL-7B	llava-mistral-7b	<u>0.754</u>	<u>0.812</u>	0.118
idefics2-8b	llava-mistral-7b	0.721	0.602	0.226

Table 4: SGPU transfers across modalities. Each row shows the SGPU trained on one modality and tested on another. **Bold** values indicates best performance.

CIFAR10 / TriviaQA					
Trained on	Tested on	AUROC↑	AUARC↑	ECE↓	
CIFAR10 / Qwen2.5-VL-3B	CIFAR10 / Qwen2.5-VL-3B	0.830	0.810	0.010	
TriviaQA / Llama-3.1-8B	CIFAR10 / Qwen2.5-VL-3B	0.775	0.745	0.122	
TriviaQA / Llama-3.1-8B	TriviaQA / Llama-3.1-8B	0.870	0.867	0.015	
CIFAR10 / Qwen2.5-VL-3B	TriviaQA / Llama-3.1-8B	0.616	0.739	0.064	
OKVQA / TriviaQA					
Trained on	Tested on	AUROC↑	AUARC↑	ECE↓	
OKVQA / idefics2-8b	OKVQA / idefics2-8b	0.753	0.798	0.150	
TriviaQA / Llama-3.1-8B	OKVQA / idefics2-8b	0.679	0.755	0.194	
TriviaQA / Llama-3.1-8B	TriviaQA / Llama-3.1-8B	0.870	0.867	0.015	
OKVQA / idefics2-8b	TriviaQA / Llama-3.1-8B	0.818	0.835	0.109	

SGPU trained with Llama-3.1-8B on **TriviaQA** and evaluated on **CIFAR10** with Qwen2.5-VL-3B achieves an AUROC of 0.775. Conversely, when SGPU is trained with Qwen2.5-VL-3B on **CIFAR10** and evaluated on **TriviaQA** with Llama-3.1-8B, it obtains an AUROC of 0.616. This difference likely stems from the fundamentally distinct nature of image classification and textual question-answering tasks, where the types of generated outputs vary significantly. SGPU shows better robustness when transferring between similar tasks across different modalities. For instance, SGPU trained with **idefics2-8b** on **OKVQA** and evaluated on **TriviaQA** with Llama-3.1-8B achieves an AUROC of 0.818. These results seem to show that SGPU may generalize beyond specific datasets, models or modalities.

5.3 Computational Cost

We report the inference time comparison of different semantic UQ methods in Table 5. In particular, we consider approaches that leverage NLI models to assess the consistency of generated answers. Since answer generation is identical

across methods, we measure only the post-hoc computational cost, i.e., the time required to compute the uncertainty metric from the sampled answers. Note that the reported differences are thus independent of the LVLM used. As shown in Table 5, SGPU achieves a lower post-hoc computational overhead compared to other approaches.

Table 5: Average inference time over 100 samples.

Method	OKVQA	VIZWIZ
SGPU	0.66 s	1.01 s
KLE-Matern	2.01 s	2.61 s
SE	0.72 s	1.04 s

6 Conclusion

Measuring semantic uncertainty in LVLMs is of critical importance for improving their reliability. In this work, we propose Semantic Gaussian Process Uncertainty (SGPU), a new Bayesian framework for quantifying semantic uncertainty in LVLMs that avoids explicit clustering and operates on the eigenspectrum of answer embeddings. Our approach provides three main advantages. First, SGPU achieves strong and often state-of-the-art performance across models and datasets on standard uncertainty-evaluation metrics such as AUROC, AUARC, and ECE, with particularly strong results on AUROC. Second, SGPU works in a fully black-box setting: it does not require access to internal model features, embeddings, or token-level probabilities. This makes it applicable to most real-world LVLMs where internal access is not available. Third, by using a Gaussian Process Classifier, SGPU can quantify its own uncertainty while predicting the uncertainty of the LVLM. This extra layer of confidence estimation makes the final predictions more stable and reliable. Overall, SGPU offers a simple, scalable, and robust way to measure uncertainty in modern LVLMs, in both white-box and black-box scenarios.

References

1. Abdaljalil, S., Kurban, H., Sharma, P., Serpedin, E., Atat, R.: Sindex: Semantic inconsistency index for hallucination detection in llms. arXiv preprint arXiv:2503.05980 (2025) [2](#), [4](#), [5](#), [3](#)
2. Abdar, M., Pourpanah, F., Hussain, S., Rezazadegan, D., Liu, L., Ghavamzadeh, M., Fieguth, P., Cao, X., Khosravi, A., Acharya, U.R., Makarenkov, V., Nahavandi, S.: A review of uncertainty quantification in deep learning: Techniques, applications and challenges. IF (2021) [1](#)
3. Aichberger, L., Schweighofer, K., Hochreiter, S.: Rethinking uncertainty estimation in natural language generation. arXiv preprint arXiv:2412.15176 (2024) [1](#), [2](#)
4. Arakelyan, E., Liu, Z., Augenstein, I.: Semantic sensitivities and inconsistent predictions: Measuring the fragility of NLI models. In: Graham, Y., Purver, M. (eds.) Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 432–444. Association for Computational Linguistics, St. Julian’s, Malta (Mar 2024). <https://doi.org/10.18653/v1/2024.eacl-long.27>, <https://aclanthology.org/2024.eacl-long.27/> [2](#), [3](#)
5. Azaria, A., Mitchell, T.: The internal state of an LLM knows when it’s lying. In: EMNLP (2023) [4](#), [6](#)
6. Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., et al.: Qwen2. 5-vl technical report. arXiv preprint arXiv:2502.13923 (2025) [10](#)
7. Band, N., Rudner, T.G.J., Feng, Q., Filos, A., Nado, Z., Dusenberry, M., Jerfel, G., Tran, D., Gal, Y.: Benchmarking bayesian deep learning on diabetic retinopathy detection tasks. In: NeurIPS Datasets and Benchmarks (2021) [8](#), [9](#)
8. Baumann, A., Li, R., Klasson, M., Mentu, S., Karthik, S., Akata, Z., Solin, A., Trapp, M.: Post-hoc probabilistic vision-language models. arXiv preprint arXiv:2412.06014 (2024) [4](#)
9. Binkowski, J., Janiak, D., Sawczyn, A., Gabrys, B., Kajdanowicz, T.J.: Hallucination detection in LLMs using spectral features of attention maps. In: EMNLP (2025) [2](#), [4](#), [3](#)
10. Blundell, C., Cornebise, J., Kavukcuoglu, K., Wierstra, D.: Weight uncertainty in neural network. In: ICML (2015) [4](#)
11. Chang, Y., He, H., Roth, D.: Popqa: A question answering benchmark for evaluating the factual consistency of language models. In: NAACL (2023) [10](#), [6](#), [11](#)
12. Chen, C., Liu, K., Chen, Z., Gu, Y., Wu, Y., Tao, M., Fu, Z., Ye, J.: INSIDE: LLMs’ internal states retain the power of hallucination detection. In: ICLR (2024) [2](#), [6](#), [8](#), [11](#), [3](#), [4](#)
13. Cohen, R., Dobler, K., Biran, E., de Melo, G.: I don’t know: Explicit modeling of uncertainty with an [idk] token. In: NeurIPS (2024) [4](#)
14. Cohen, R., Hamri, M., Geva, M., Globerson, A.: LM vs LM: Detecting factual errors via cross examination. In: EMNLP (2023) [4](#)
15. Daxberger, E., Kristiadi, A., Immer, A., Eschenhagen, R., Bauer, M., Hennig, P.: Laplace redux-effortless bayesian deep learning. In: NeurIPS (2021) [4](#)
16. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: CVPR (2009) [6](#)
17. Ettori, D., Darabi, N., Tayebati, S., Krishnan, R., Subedar, M., Tickoo, O., Trivedi, A.R.: Eigentrack: Spectral activation feature tracking for hallucination and out-of-distribution detection in llms and vlms. arXiv preprint arXiv:2509.15735 (2025) [4](#), [6](#)

18. Farquhar, S., Kossen, J., Kuhn, L., Gal, Y.: Detecting hallucinations in large language models using semantic entropy. *Nature* (2024) 2, 4, 5, 8, 9, 11, 6
19. Franchi, G., Bursuc, A., Aldea, E., Dubuisson, S., Bloch, I.: Tradi: Tracking deep neural network weight distributions. In: *ECCV* (2020) 4
20. Gabetni, F., Curci, G., Pilzer, A., Roy, S., Ricci, E., Franchi, G.: Ensembling pruned attention heads for uncertainty-aware efficient transformers. In: *ICLR* (2026) 4
21. Gal, Y., Ghahramani, Z.: Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In: *ICML* (2016) 1, 4
22. Gao, X., Zhang, J., Mouatadid, L., Das, K.: SPUQ: Perturbation-based uncertainty quantification for large language models. In: *EACL* (2024) 4
23. Grewal, Y.S., Bonilla, E.V., Bui, T.D.: Improving uncertainty quantification in large language models via semantic embeddings. *arXiv preprint arXiv:2410.22685* (2024) 2, 4, 3
24. Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q.: On calibration of modern neural networks. In: *ICML* (2017) 11, 7
25. Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al.: Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature* (2025) 1
26. Gurari, D., Li, Q., Stangl, A.J., Guo, A., Lin, C., Grauman, K., Luo, J., Bigham, J.P.: Vizwiz grand challenge: Answering visual questions from blind people. In: *CVPR* (2018) 10, 5
27. He, P., Liu, X., Gao, J., Chen, W.: {DEBERTA}: {DECODING}-{enhanced} {bert} {with} {disentangled} {attention}. In: *International Conference on Learning Representations* (2021), <https://openreview.net/forum?id=XPZiaotutsD> 4, 8
28. Hendrycks, D., Gimpel, K.: A baseline for detecting misclassified and out-of-distribution examples in neural networks. In: *ICLR* (2017) 11, 6
29. Hernandez-Lobato, J.M., Adams, R.: Probabilistic backpropagation for scalable learning of bayesian neural networks. In: *ICML* (2015) 1
30. Howard, J.: Imagenette: A smaller subset of 10 easily classified classes from imagenet. <https://github.com/fastai/imagenette> (2019) 10
31. Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., Liu, T.: A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *TIS* (2025) 1
32. Janiak, D., Binkowski, J., Sawczyn, A., Gabrys, B., Shwartz-Ziv, R., Kajdanowicz, T.J.: The illusion of progress: Re-evaluating hallucination detection in LLMs. In: *EMNLP* (2025) 2, 6, 8, 9, 11, 3, 4
33. Janiak, D., Binkowski, J., Sawczyn, A., Gabrys, B., Shwartz-Ziv, R., Kajdanowicz, T.J.: The illusion of progress: Re-evaluating hallucination detection in llms. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. pp. 34716–34733 (2025) 8
34. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. *CS* (2023) 1
35. Ji, Z., Yu, L., Koishekenov, Y., Bang, Y., Hartshorn, A., Schelten, A., Zhang, C., Fung, P., Cancedda, N.: Calibrating verbal uncertainty as a linear feature to reduce hallucinations. *arXiv preprint arXiv:2503.14477* (2025) 2, 5
36. Joo, M., Cho, H.: Cleanse: Uncertainty estimation approach using clustering-based semantic consistency in LLMs. In: *ACL Workshops* (2025) 4
37. Joshi, M., Choi, E., Weld, D.S., Zettlemoyer, L.: Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In: *ACL* (2017) 10, 6, 11

38. Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., et al.: Language models (mostly) know what they know. arXiv preprint arXiv:2207.05221 (2022) 4, 5, 6
39. Krizhevsky, A., Hinton, G., et al.: Learning multiple layers of features from tiny images (2009) 10, 6
40. Kuhn, L., Gal, Y., Farquhar, S.: Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In: ICLR (2023) 4, 5, 8, 9, 11, 6
41. Kuhn, L., Gal, Y., Farquhar, S.: Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In: ICLR (2023) 11, 6
42. Lakshminarayanan, B., Pritzel, A., Blundell, C.: Simple and scalable predictive uncertainty estimation using deep ensembles. In: NeurIPS (2017) 1, 4
43. Lau, G.K.R., Dao, H., Lin, N.K.H., Low, B.K.H.: Uncertainty quantification for multimodal large language models with coherence-adjusted semantic volume (2025), <https://openreview.net/forum?id=c9TWekZQR4> 4, 6, 11
44. Lau, J.J., Gayen, S., Ben Abacha, A., Demner-Fushman, D.: A dataset of clinically generated visual questions and answers about radiology images. Scientific data (2018) 10, 5
45. Laurençon, H., Tronchon, L., Cord, M., Sanh, V.: What matters when building vision-language models? In: NeurIPS (2024) 10
46. Laurent, O., Aldea, E., Franchi, G.: A symmetry-aware exploration of bayesian neural network posteriors. arXiv preprint arXiv:2310.08287 (2023) 4
47. Laurent, O., Lafage, A., Tartaglione, E., Daniel, G., Martinez, J.M., Bursuc, A., Franchi, G.: Packed-ensembles for efficient uncertainty estimation. arXiv preprint arXiv:2210.09184 (2022) 4
48. Lee, S., Kim, H., Hwang, J., Park, E., Ok, J.: Efficient latent semantic clustering for scaling test-time computation of llms (2025), <https://arxiv.org/abs/2506.00344> 2, 4, 5, 3
49. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive nlp tasks. In: NeurIPS (2020) 3
50. Li, K., Patel, O., Viégas, F., Pfister, H., Wattenberg, M.: Inference-time intervention: Eliciting truthful answers from a language model. In: NeurIPS (2023) 4
51. Li, L., Lei, J., Gan, Z., Liu, J.: Adversarial vqa: A new benchmark for evaluating the robustness of vqa models. In: ICCV (2021) 10, 5
52. Li, X., Yu, Z., Zhang, Z., Zhuang, Y., Shah, S., Sadagopan, N., Beniwal, A.: Semantic volume: Quantifying and detecting both external and internal uncertainty in llms. arXiv preprint arXiv:2502.21239 (2025) 4, 6
53. Lin, S., Hilton, J., Evans, O.: Teaching models to express their uncertainty in words. TMLR (2022) 4
54. Lin, Z., Trivedi, S., Sun, J.: Generating with confidence: Uncertainty quantification for black-box large language models. TMLR (2024) 4, 5, 6
55. Liu, H., Xue, W., Chen, Y., Chen, D., Zhao, X., Wang, K., Hou, L., Li, R., Peng, W.: A survey on hallucination in large vision-language models. arXiv preprint arXiv:2402.00253 (2024) 1
56. Liu, H., Li, C., Li, Y., Lee, Y.J.: Improved baselines with visual instruction tuning. In: CVPR (2024) 10
57. MacKay, D.J.: Probable networks and plausible predictions-a review of practical bayesian methods for supervised neural networks. Network: computation in neural systems (1995) 4

58. Mackay, D.J.C.: Bayesian methods for adaptive models. CalTech (1992) 4
59. Maddox, W.J., Izmailov, P., Garipov, T., Vetrov, D.P., Wilson, A.G.: A simple baseline for bayesian uncertainty in deep learning. NeurIPS (2019) 4
60. Malinin, A., Gales, M.: Uncertainty estimation in autoregressive structured prediction. In: ICLR (2021) 1, 4
61. Manakul, P., Liusie, A., Gales, M.: SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In: EMNLP (2023) 4, 8, 9
62. Marino, K., Rastegari, M., Farhadi, A., Mottaghi, R.: Ok-vqa: A visual question answering benchmark requiring external knowledge. In: CVPR (2019) 10, 5
63. Murphy, K.P.: Machine learning: a probabilistic perspective. MIT press (2012) 3
64. Murray, K., Chiang, D.: Correcting length bias in neural machine translation. In: MT (2018) 4
65. Neal, R.M.: Bayesian learning for neural networks. Springer Science & Business Media (2012) 4
66. Nikitin, A., Kossen, J., Gal, Y., Marttinen, P.: Kernel language entropy: Fine-grained uncertainty quantification for llms from semantic similarities. In: NeurIPS (2024) 2, 4, 5, 11, 6
67. Novikov, A., Vū, N., Eisenberger, M., Dupont, E., Huang, P.S., Wagner, A.Z., Shirobokov, S., Kozlovskii, B., Ruiz, F.J., Mehrabian, A., et al.: Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131 (2025) 1
68. Orgad, H., Toker, M., Gekhman, Z., Reichart, R., Szpektor, I., Kotek, H., Belinkov, Y.: LLMs know more than they show: On the intrinsic representation of LLM hallucinations. In: ICLR (2025) 4, 6
69. Qiu, X., Miikkulainen, R.: Semantic density: Uncertainty quantification for large language models through confidence measurement in semantic space. In: NeurIPS (2024) 4
70. Rasmussen, C.E., Williams, C.K.I.: Gaussian Processes for Machine Learning. The MIT Press (2005) 9, 10
71. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. arXiv preprint arXiv:1908.10084 (2019) 11, 3, 4, 8
72. Ren, J., Luo, J., Zhao, Y., Krishna, K., Saleh, M., Lakshminarayanan, B., Liu, P.J.: Out-of-distribution detection and selective generation for conditional language models. In: ICLR (2023) 4
73. Ritter, H., Botev, A., Barber, D.: A scalable laplace approximation for neural networks. In: ICLR (2018) 4
74. Shwartz-Ziv, R., Balestrieri, R., Kawaguchi, K., Rudner, T.G.J., LeCun, Y.: An information theory perspective on variance-invariance-covariance regularization. In: NeurIPS (2023) 6
75. Sriramanan, G., Bharti, S., Sadasivan, V.S., Saha, S., Kattakinda, P., Feizi, S.: Llm-check: Investigating detection of hallucinations in large language models. In: NeurIPS (2024) 6
76. Team, G.: Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context (2024), <https://arxiv.org/abs/2403.05530> 1
77. team, M.: The llama 3 herd of models (2024), <https://arxiv.org/abs/2407.21783> 11
78. team, O.: Gpt-4 technical report (2024), <https://arxiv.org/abs/2303.08774> 1
79. team, O.: gpt-oss-120b and gpt-oss-20b model card (2025), <https://arxiv.org/abs/2508.10925> 1

80. Wei, H., Sun, Y., Li, Y.: Deepseek-ocr: Contexts optical compression. arXiv preprint arXiv:2510.18234 (2025) 10
81. Wen, Y., Tran, D., Ba, J.: Batchensemble: an alternative approach to efficient ensemble and lifelong learning. In: International Conference on Learning Representations (2020), <https://openreview.net/forum?id=Sk1f1yrYDr> 4
82. Williams, C.K., Rasmussen, C.E.: Gaussian processes for machine learning, vol. 2. MIT press Cambridge, MA (2006) 4
83. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T.L., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M.: Transformers: State-of-the-art natural language processing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 38–45. Association for Computational Linguistics, Online (Oct 2020), <https://www.aclweb.org/anthology/2020.emnlp-demos>. 6 7
84. Zhang, T., Shi, H., Wang, Y., Wang, H., He, X., Li, Z., Chen, H., Han, L., Xu, K., Zhang, H., et al.: Token-level uncertainty estimation for large language model reasoning. arXiv preprint arXiv:2505.11737 (2025) 4
85. Zhouyin, Z., Liu, D.: Understanding neural networks with logarithm determinant entropy estimator. arXiv preprint arXiv:2105.03705 (2021) 6
86. Zhouyin, Z., Liu, D.: Understanding neural networks with logarithm determinant entropy estimator. Neurocomputing (2025) 6

Improving Semantic Uncertainty Quantification in LVLMs with Semantic Gaussian Processes –Supplementary Material–

A Proofs of Section 4.1

This section provides the proofs of Lemma 1 and Lemma 2 discussed in Section 4.1. We start by introducing the following lemma.

Lemma 3. *Let $\Phi^* = [\phi_1^*, \dots, \phi_N^*] \in \mathbb{R}^{p \times N}$ denote the embedding matrix of an ideal sentence encoder $E^*(\cdot)$ defined as in Assumption 1. Let $\mathcal{E} := \{\mathbf{e}_1, \dots, \mathbf{e}_p\}$ the standard basis of $\mathbb{R}^p$. Then, there exists an orthogonal matrix $\mathbf{P} \in \mathbb{R}^{p \times p}$ such that*

$$\Phi^* = \mathbf{P}\mathbf{U}, \quad (13)$$

where $\mathbf{U} = [\mathbf{u}_1 \ \mathbf{u}_2 \ \dots \ \mathbf{u}_N] \in \mathbb{R}^{p \times N}$ with $\mathbf{u}_i \in \mathcal{E}$ for all i .

Proof. Under Assumption 1, there exists a unique set $\mathcal{V} := \{\mathbf{v}_1, \dots, \mathbf{v}_k\} \in \{\phi_1^*, \dots, \phi_N^*\}$ of orthonormal vectors in $\mathbb{R}^p$. The set $\mathcal{V}$ can be completed with $p - k$ additional orthonormal vectors to form a basis $\mathcal{B}$ of $\mathbb{R}^p$. Let $\mathbf{P} \in \mathbb{R}^{p \times p}$ denote the matrix whose columns are the vectors $\mathcal{B}$. $\mathbf{P}$ is an orthonormal matrix and depicts the change of basis matrix from $\mathcal{B}$ to the standard basis $\mathcal{E}$ of $\mathbb{R}^p$ ($\mathbf{P} = \mathbf{P}\mathbf{I}_p$). We obtain $\mathbf{U} = \mathbf{P}^T \Phi^*$, which concludes the proof.

Leveraging Lemma 3 and the Weinstein–Aronszajn identity, the following Lemma 1 shows that the eigenvalues of the Gram matrix $\Sigma^* = \Phi^{*T} \Phi^* \in \mathbb{R}^{N \times N}$ obtained from an ideal sentence encoder capture meaningful information about the semantic consistency among the generated answers in $\mathcal{Y}$ across meaning.

Lemma 1 (Ideal Semantic Consistency Vector). *Let $\lambda^* = [\lambda_1^*, \dots, \lambda_N^*]^T \in \mathbb{R}^N$ be the vector of eigenvalues of $\Sigma^* = \Phi^{*T} \Phi^*$ ordered such that $\lambda_1^* \geq \dots \geq \lambda_N^* \geq 0$. Then, λ^* depicts the repartition of generated answers $\mathcal{Y}$ across meanings, where:*

1. *The sum of eigenvalues equals the number of generated answers N , i.e., $\sum_{i=1}^N \lambda_i^* = N$.*
2. *The number of nonzero eigenvalues in λ^* corresponds to the number of distinct semantic meanings present in the generated answers $\mathcal{Y}$, i.e., the number of ideal distinct semantic clusters.*
3. *Each nonzero λ_i^* is equal to the number of generated answers in $\mathcal{Y}$ that share a same semantic meaning, i.e., the size of an ideal semantic cluster.*

Proof. Let $\mathcal{E} := \{e_1, \dots, e_p\}$ the standard basis of $\mathbb{R}^p$. From Lemma 3, there exists an orthogonal matrix $\mathbf{P} \in \mathbb{R}^{p \times p}$ such that

$$\Phi^* = \mathbf{P}\mathbf{U},$$

where $\mathbf{U} = [\mathbf{u}_1 \mathbf{u}_2 \dots \mathbf{u}_N] \in \mathbb{R}^{p \times N}$ with $\mathbf{u}_i \in \mathcal{E}$ for all i . From the Weinstein–Aronszajn identity, the matrices

$$\Sigma^* = \Phi^{*T} \Phi^* = \mathbf{U}^T \underbrace{\mathbf{P}^T \mathbf{P}}_{=\mathbf{I}_p} \mathbf{U} = \mathbf{U}^T \mathbf{U} \in \mathbb{R}^{N \times N}$$

and

$$\mathbf{D} = \mathbf{U}\mathbf{U}^T \in \mathbb{R}^{p \times p}$$

share the same nonzero eigenvalues. Let $i, j \in [p]$. We have

$$D_{ij} = [\mathbf{U}\mathbf{U}^T]_{ij} = \sum_{k=1}^N \mathbf{U}_{ik} \mathbf{U}_{jk} = \sum_{k=1}^N [\mathbf{u}_k]_i [\mathbf{u}_k]_j.$$

For all k , $\mathbf{u}_k \in \mathcal{E}$ is a vector in the standard base of $\mathbb{R}^p$, i.e., a vector which has a single element equal to 1 and all other components equal to 0. Therefore, for all k and $i \neq j$, we have $[\mathbf{u}_k]_i [\mathbf{u}_k]_j = 0$. For all $i \neq j$, we deduce that $D_{ij} = [\mathbf{U}\mathbf{U}^T]_{ij} = 0$ and that $\mathbf{D}$ is a diagonal matrix. Nonzero eigenvalues of Σ^* are thus diagonal elements of $\mathbf{D}$. Furthermore, for all i , we have

$$D_{ii} = \sum_{k=1}^N [\mathbf{u}_k]_i^2 = \sum_{k=1}^N [\mathbf{u}_k]_i = c_i,$$

where c_i depicts the number of occurrences of e_i in $\mathbf{U}$.

Leveraging the Weyl’s inequality, Lemma 2 depicts how the eigenvalues of the Gram matrix $\Sigma = \Phi^T \Phi$ (equation 8) derived from an external sentence encoder approximate the ideal semantic consistency vector λ^* .

Lemma 2. *Let $\lambda \in \mathbb{R}^N$ be the vector of eigenvalues of Σ ordered such that $\lambda_1 \geq \dots \geq \lambda_N \geq 0$. For all $i \in [N]$, we have*

$$\delta_i = \lambda_i - \lambda_i^* \in [-\sqrt{N(N-1)}\epsilon, \sqrt{N(N-1)}\epsilon],$$

where $\epsilon = \max_{i,j} |\phi_i^T \phi_j - \phi_i^{*T} \phi_j^*| \in [0, 2]$. Besides, $\sum_{i=1}^N \delta_i = 0$.

Proof. Let $\Delta = \Sigma - \Sigma^* \in \mathbb{R}^{N \times N}$ and $\epsilon = \max_{i,j} |\Delta_{ij}|$. Note that Δ is symmetric and that $\text{Tr}(\Delta) = 0$. We can observe that

$$\Sigma = \Sigma^* + [\Sigma - \Sigma^*] = \Sigma^* + \Delta.$$

From equation above and the Weyl’s inequality, for all $i \in [N]$, we have

$$|\delta_i| = |\lambda_i - \lambda_i^*| \leq \max(|\lambda_N(\Delta)|, \lambda_1(\Delta)) = \|\Delta\|_2$$

where $\lambda_N(\Delta) \in \mathbb{R}$ and $\lambda_1(\Delta) > 0$ denote the minimum et maximum eigenvalues of Δ . From the equivalence norm, we have

$$\|\Delta\|_2 \leq \|\Delta\|_F,$$

where $\|\cdot\|_F$ is the Frobenius norm. By construction of Σ and Σ^* , all diagonal elements of $\Delta = \Sigma - \Sigma^*$ are equal to zeros. We obtain thus:

$$\|\Delta\|_F = \sqrt{\sum_{i=1}^N \sum_{j=1}^N \Delta_{ij}^2} = \sqrt{\sum_{i=1}^N \sum_{j \neq i}^N \Delta_{ij}^2} \leq \sqrt{N(N-1)\epsilon^2} = \sqrt{N(N-1)}\epsilon.$$

Finally, we have $\sum_{i=1}^N \delta_i = \sum_{i=1}^N \lambda_i - \lambda_i^* = \text{Tr}(\Sigma) - \text{Tr}(\Sigma^*) = N - N = 0$.

B Impacts of Embeddings on the Semantic Consistency

B.1 Semantic Similarities in UQ

As mentioned in Section 3, semantic UQ methods estimate uncertainty by computing semantic similarities between generated answers either to perform semantic clustering or to directly derive a semantic UQ score, e.g., the semantic volume. These methods can rely on two types of semantic spaces for such comparisons.

Natural Language Inference (NLI). In practice, semantic similarities are typically obtained using predictions from NLI models. However, despite their popularity, NLI models often struggle to capture the full range of semantic properties in text and are highly sensitive to minor variations in wording, additional correct information, or the presence of non-essential words in generated responses [4, 23] as illustrated in Table A.6.

Semantic Embeddings. Another line of work explores computing semantic similarities from embeddings derived from the hidden states of LVLMs [9, 12, 32, 48]. Yet, such representations are not specifically designed for this purpose, and these methods are limited to white-box access to the underlying LLM. Following prior works [1, 23], we propose instead to leverage external sentence encoders that are specifically trained to derive semantically meaningful sentence [71] and are used in applications such as retrieval [49], classification [71] or clustering [1]. In the following section, we indeed observe that embeddings of generated answers derived from an external sentence encoder capture the semantic consistency among those answers more effectively than hidden representations of LVLMs.

Table A.6: Comparison of bidirectional entailment and cosine similarity for assessing semantic equivalence [23]. Bidirectional entailment scores are obtained using DeBERTaLarge [27] NLI model, while cosine similarity is computed with sentence-BERT [71].

Context	Responses	Bidirectional Entailment	Cosine Similarity
What is the primary function of the mitochondria in cells?	1. The mitochondria produce energy for the cells. 2. Mitochondria provides energy to cells in the body.	False	0.974
What happens when you heat ice?	1. Heating ice will eventually boil after becoming water. 2. When ice is heated, it melts into water before boiling.	False	0.893
What do mammals have in common?	1. Mammals are warm-blooded and have hair or fur. 2. All mammals (like humans and dogs) are warm-blooded creatures with hair.	False	0.927

Table A.7: Impacts of Embeddings on the Semantic Consistency Vector over 100 samples from the **OKVQA** dataset. For each sample we prompt **Qwen2.5-VL-7B** to generate 10 paraphrases of the reference answer and 5 paraphrases of a semantically distinct answer to get two clusters of 10 and 5 responses. Report of the average of the two largest eigenvalues λ_1 and λ_2 of the Gram matrices computed for each embedding type across all samples and of the maximum semantic similarity error ϵ (Lemma 2).

Sentence Embedding	λ_1	λ_2	ϵ
Ideal Embedding	10.0	5.00	0.00
External Embedding	8.41	3.96	0.49
Internal Embedding (layer 16)	11.73	1.85	0.67

B.2 Impacts of Embeddings on the Semantic Consistency Vector

In Section 4.1 of the main paper, we introduce the semantic consistency vector $\lambda \in \mathbb{R}^N$ defined as the eigenvalues of the Gram matrix computed from the embeddings of generated answers. This vector provides a compact yet rich descriptor of semantic consistency of generated answers. In particular, Lemma 1 interprets the eigenvalues $\lambda^* \in \mathbb{R}^N$ of the Gram matrix derived from ideal embeddings of generated answers (Assumption 1) as the repartition of generated answers across their meanings. In practice, however, the embeddings are obtained from hidden representations or external models, yielding only an approximation of the Gram matrix associated with an ideal encoder. As highlighted by Lemma 2, such approximations can perturb the information captured by the eigenvalues $\lambda \in \mathbb{R}^N$ regarding the distribution of generated answers, thereby limiting the direct use of these values in UQ methods, e.g., by computing the semantic volume [12] or the von Neumann entropy [32].

Impact of Embeddings. To investigate the impact of embeddings on the information captured by the semantic consistency vector $\lambda = [\lambda_1 \dots \lambda_N]^T \in \mathbb{R}^N$ (defined in Lemma 2), we conduct a pilot study using 100 samples from the **OKVQA** dataset. In particular, for each sample, we prompt **Qwen2.5-VL-7B** to generate

10 paraphrases of the reference answer and 5 paraphrases of a semantically distinct answer to get two clusters of 10 and 5 responses, respectively. We evaluate two kinds of sentence embeddings: one derived from the hidden representation of the LVLM at an intermediate layer³, and another from the external sentence encoder `all-MiniLM-L6-v2` used in the main experiments. Table A.7 reports the average of the two largest eigenvalues of the Gram matrices computed for each embedding type across all samples. According to Lemma 1, the eigenspectrum of the Gram matrix obtained from an ideal sentence encoder should reflect the distribution of sentences across their meanings. In this pilot study, the expected eigenspectrum should contain two nonzero eigenvalues equal to 10 and 5. Our results indicate that semantic consistency vectors derived from `all-MiniLM-L6-v2` capture information about the distribution of generated answers across different meanings more effectively than those based on hidden representations. Although semantic consistency vectors derived from external encoders provide a more accurate representation of the underlying semantic distribution, the information they convey remains perturbed, which limits their direct applicability for computing a UQ score. These findings motivate the introduction of SGPU, which more effectively leverages the semantic consistency information encoded in the eigenspectrum despite the approximation errors introduced by the external sentence encoder.

C Experimental Settings

In this section, we provide additional details about the experimental protocol used in Section 5.

C.1 Datasets

We evaluate SGPU on Visual Question Answering (VQA), image classification, and textual Question Answering (QA) tasks.

Visual Question-Answering. For VQA, we consider four datasets: **ADVQA** [51], which consists of a training set of 6,000 samples and a test set of 2,000 samples both drawn from the original training set; **VQARAD** [44], which consists of a training set of 2,000 samples from the original training set and a test set of 500 samples from the original test set; **OKVQA** [62], which consists of a training set of 9,000 samples drawn from the original training set and a test set of 5,000 samples drawn from the original validation set; and **VizWiz** [26], which consists of a training set of 10,000 samples and a test set of 2,000 samples both drawn from the original training set.

³ We choose the intermediate layer that provides the best approximation of the ideal Gram matrix.

Image Classification. For image classification, we use the **CIFAR-10** [39] and **Imagenette** [16] datasets, each split into a training set of 10,000 samples drawn from the original training data and a test set of 5,000 samples taken from the original validation set.

Question-Answering. For text-based QA, we use the **TriviaQA** [37] and **PopQA** [11] datasets, each split into a training set of 10,000 samples drawn from the original training data and a test set of 1,000 samples taken from the original test set.

C.2 Baselines

We compare SGPU with popular uncertainty-based methods using both the logits and hidden representations of LVLMS.

Logit-based Methods. For logit-based measures, we consider the following: **Predictive Entropy (PE)** [18, 38, 54] defined in equation 3; **Semantic Entropy (SE)** [18, 40] defined in equation 5; **Discrete Semantic Entropy (DSE)** [18, 40] defined in equation 6; **Kernel Language Entropy (KLE)** [66], which captures semantic similarities by applying a distance measure in the space of the generated answers. In particular, KLE uses an external NLI model to construct a semantic graph over the generated responses, and then computes the von Neumann entropy of a graph kernel formed by the combination of the graph Laplacian with a predefined kernel.

Latent-based Methods. We also examine semantic UQ methods that exploit dense semantic information retained within the internal states of LVLMS. In particular, we consider: **Cos Eigenscore** [43, 52] and **Cov Eigenscore** [12], which are semantic volume-based measures (see equation 7). These methods compute the semantic volume using the cosine similarity matrix and the empirical covariance matrix of the feature representations, respectively. We also include **UMPIRE** [43], which combines the semantic volume with the probabilities of generated answers.

C.3 Metrics

AUROC (Area Under the ROC Curve). We assign a probability score to each example for belonging to the positive class (positive reflects that the LVLMS is certain about its answer). AUROC [28] measures how well these scores rank true positives above true negatives. It is the chance that a randomly chosen positive example receives a higher score than a randomly chosen negative one. Higher AUROC means better class separation.

AUARC (Area Under Accuracy-Retention Curve). Each prediction has an associated uncertainty. We sort predictions from most confident to least confident and progressively discard (abstain on) the most uncertain ones. For every retention level (fraction kept), we compute the accuracy on the retained subset. AUARC [41] summarizes how accuracy improves as we keep only confident predictions. Higher AUARC means uncertainty is useful for selective prediction.

Table A.8: SGPU is robust to the choice of kernel hyperparameters. Sensitivity analysis of the impact of kernel hyperparameters on SGPU performance on the **VizWiz** dataset using **Qwen2.5-VL-7B**.

Kernel Hyperparameters	AUROC	AUARC	ECE
Matern (length_scale=1 nu=1.5)	0.818	0.808	0.021
Matern (length_scale=1 nu=2.5)	0.818	0.803	0.014
Matern (length_scale=1 nu=0.5)	0.814	0.807	0.018
Matern (length_scale=0.5 nu=2.5)	0.810	0.797	0.012
Matern (length_scale=0.5 nu=1.5)	0.810	0.797	0.016
Matern (length_scale=0.5 nu=0.5)	0.811	0.797	0.028
RBF (length_scale=1)	0.817	0.805	0.018
RBF (length_scale=0.5)	0.812	0.798	0.018
RBF (length_scale=2)	0.819	0.806	0.022
RBF (length_scale=2.5)	0.819	0.806	0.027
RationalQuadratic (alpha=0.1 length_scale=0.5)	0.814	0.803	0.017
RationalQuadratic (alpha=0.2 length_scale=0.5)	0.814	0.803	0.017
RationalQuadratic (alpha=0.1 length_scale=1)	0.815	0.803	0.018
RationalQuadratic (alpha=0.2 length_scale=1)	0.816	0.804	0.018

ECE (Expected Calibration Error). Each prediction has a confidence value. We group predictions into bins of similar confidence and compare, within each bin, the average confidence to the actual fraction that is correct. ECE [24] is the average mismatch across bins. Lower ECE means the reported confidences align well with true correctness (the model is better calibrated).

C.4 Sensitivity to Design Choices

Sensitivity to Kernel Choices. In the experiments described in Section 5, SGPU uses a Matérn kernel with the default `scikit-learn` parameters (`length_scale=1.0`, $\nu = 1.5$). As summarized in Table A.8, the empirical results show that SGPU is robust to the choice of kernel hyperparameters and does not require specific tuning.

Sensitivity to External Encoder Choices. Similarly, we examine the impact of the choice of the external sentence encoder used by SGPU. Experiments depicted in Table A.9 show that SGPU remains robust to the choice of external sentence embedding. In our implementation, SGPU uses the `all-MiniLM-L6-v2` encoder, which provides a good trade-off between efficiency and representation quality.

C.5 Other Experimental Settings

For our experiments, we use the Transformer library [83]. For each sample, we sample 20 candidate answers using `top_p=0.9`, `top_k=50`, and `temperature=1.0`. All models perform inference in `torch.float16` precision on NVIDIA

Table A.9: SGPU is robust to the choice of the external sentence encoder. Sensitivity analysis of the impact of different external sentence encoders on SGPU performance on the **VizWiz** dataset using **Qwen2.5-VL-7B**.

External Encoder	AUROC	AUARC	ECE
all-MiniLM-L6-v2	0.818	0.808	0.021
all-MiniLM-L12-v2	0.814	0.802	0.020
modernbert-embed-large	0.820	0.805	0.022

H100 NVL GPUs. We use **all-MiniLM-L6-v2** [71] for sentence embedding with mean pooling and use the Deberta-large for NLI [27].

D Truthfulness Labeling

Let $\mathbf{x}^{(i)}$ denote a multimodal input (question+image) and let $\mathcal{Y}^{(i)} := \{\mathbf{y}_i^{(j)}\}_{j=1}^{20}$ be the set of 20 candidate answers generated by a LVLM given $\mathbf{x}^{(i)}$, as described in Section 4.2. Following a standard protocol in the UQ literature [7, 18, 32, 40, 61], we evaluate the correctness $l^{(i)}$ of each generated response $\mathbf{y}_i^{(j)}$ with respect to a reference answer $\bar{\mathbf{y}}_i$ using an LLM-as-judge approach. This approach ensures a fair comparison with prior work, achieves better performance than other metrics (e.g., ROUGE score), and shows strong agreement with human judgments [33]. We use **Llama-3.1-8B** to compare each output $\mathbf{y}_i^{(j)}$ with the reference answer $\bar{\mathbf{y}}_i$ and assign a binary score $l_j^{(i)} \in \{0, 1\}$ to indicate whether $\mathbf{y}_i^{(j)}$ is correct. In particular, we use the following evaluation prompt:

Prompt for Evaluating Truthfulness of Generated Answers

We are assessing the quality of answers to the following question:
[question].
The expected answer is: [correct answers].
The proposed answer is: [predicted answer].
Within the context of the question, does the proposed answer mean the same as the expected answer?
Respond only with yes or no.
Response:

The truthfulness label $l^{(i)}$ that indicate the correctness of generated answers $\mathcal{Y}^{(i)}$ is then defined as the most frequent label among $\{l_1^{(i)}, \dots, l_{20}^{(i)}\}$. Figure A.3 illustrates the labeling strategy with two samples from the **OKVQA** dataset and candidate answers generated by **Qwen2.5-VL-3B**.

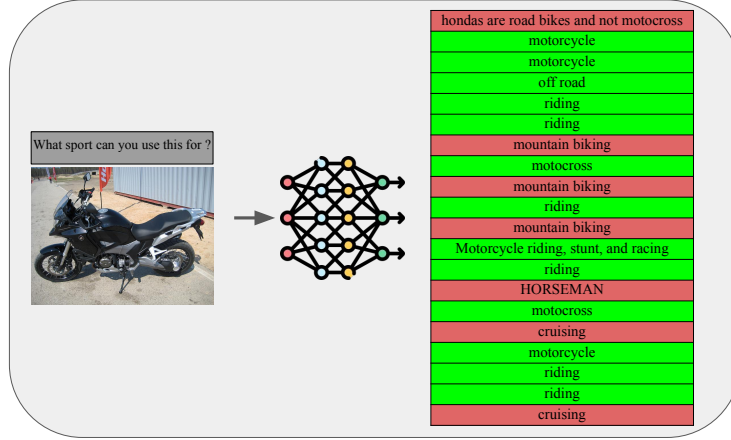
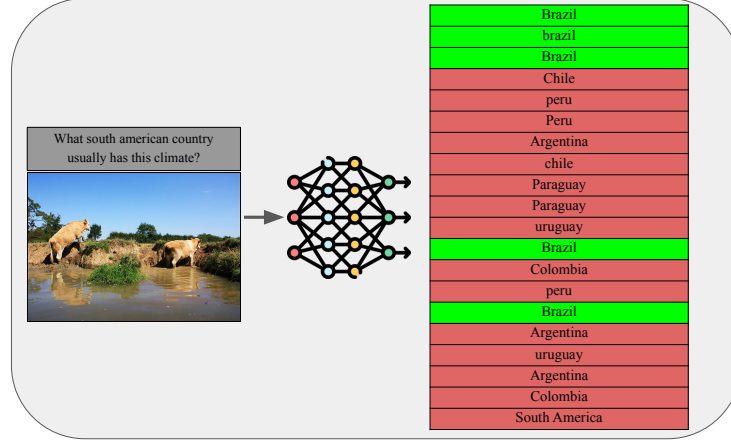
(a) Generated answers are mostly correct ($l^{(i)} = 1$)(b) Generated answers are mostly incorrect ($l^{(i)} = 0$)

Fig. A.3: Illustration of the truthfulness labeling strategy using two examples from the **OKVQA** dataset: one where the truthfulness label $l^{(i)} = 1$ and another where $l^{(i)} = 0$. Each generated answer is compared to a reference answer using an LLM-as-judge approach. Answers depicted in **green** ($l_j^{(i)} = 1$) are correct, whereas those in **red** are incorrect ($l_j^{(i)} = 0$).

E Motivations Behind the Gaussian Process

As mentioned in Section 4, the use of a Gaussian Process Classifier (GPC) is motivated by its ability to provide direct access to internal uncertainty estimates (through the predictive standard deviation), its efficiency with limited training data, and its good calibration.

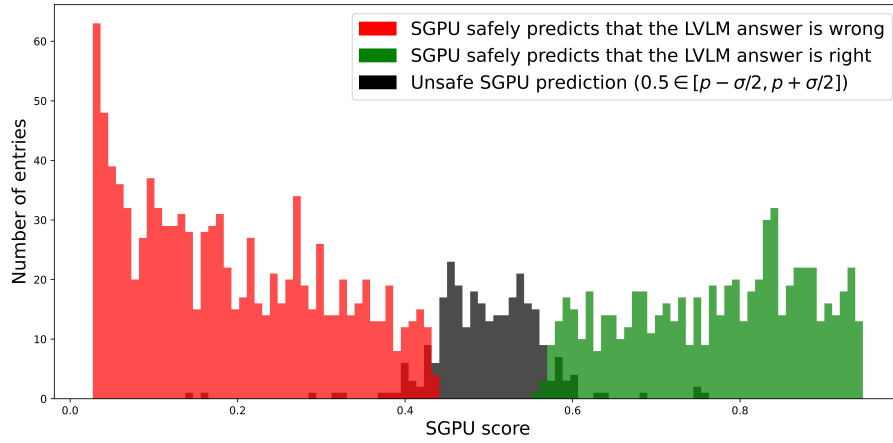


Fig. A.4: Detection of Unsafe SGPU Predictions with `llava-mistral-7b` on **VIZWIZ**. Unsafe SGPU predictions are defined when $0.5 \in [p - \sigma/2, p + \sigma/2]$, where p is the SGPU score and σ is its associated predicted standard deviation.

E.1 SGPU Internal Uncertainty

For each prediction, a GPC provides two values: the class probability and the corresponding predictive standard deviation. In the SGPU framework, the GPC returns the probability that the generated answers are similar. Predicted probabilities close to 1 or 0 indicate thus high confidence, whereas values near 0.5 correspond to regions of uncertainty. A straightforward approach to exclude uncertain predictions is to discard samples whose probabilities are close to 0.5. However, SGPU adopts a more principled approach to quantifying predictive uncertainty by leveraging the predictive standard deviation, which captures the internal uncertainty for each prediction. A prediction is flagged as *unsafe* if adding or subtracting half of the predictive standard deviation to the predicted probability changes the resulting class label. Figure A.4 depicts this classification strategy on **VizWiz** with `llava-mistral-7b`, where unsafe predictions are highlighted in black. To verify that these predictions are indeed unreliable, we compute the AUROC (using the same setup as in the main experiment) before and after removing the unsafe entries. As expected, the AUROC increases from 0.828 to 0.853 after filtering these entries, confirming that our criterion effectively identifies uncertain predictions.

Table A.10: SGPU requires only a few training samples. Evaluation of SGPU with varying number of training samples, where SGPU n denotes SGPU trained with only n samples. **Bold** values indicate best performance.

Dataset	Method	llava-mistral-7b		
		AUROC	AUARC	ECE
VizWiz	SE	0.790	0.671	0.254
	DSE	0.791	0.666	0.276
	Regular	0.755	0.650	0.046
	KLE-Heat	0.726	0.632	0.218
	KLE-Matern	0.764	0.657	0.046
	UMPIRE	0.789	0.627	0.182
	Cos Eigenscore	0.760	0.580	0.100
	Cov Eigenscore	0.701	0.546	0.092
	SGPU (ours)	0.828	0.814	0.037
	SGPU 1000	0.816	0.804	0.039
	SGPU 500	0.811	0.796	0.045
	SGPU 200	0.800	0.780	0.042
	SGPU 100	0.773	0.759	0.051

E.2 SGPU Training

Unlike popular semantic UQ methods (Section 3), SGPU requires training. Nevertheless, Gaussian Processes are well known for their strong performance even with limited training data as illustrated by Table A.10. The results show that SGPU maintains competitive performance even with a very small number of training samples. Remarkably, with only 200 training samples, it still outperforms existing state-of-the-art methods. These findings confirm the applicability and portability of SGPU.

F Additional Experiments

F.1 Additional Experiments on LLM

We also evaluate SGPU on text-only tasks and LLM with Llama-3.1-8B-Instruct [77], using the same experimental setup as in the multimodal experiments. In particular, we conduct our experiments on two question-answering (QA) benchmarks: **TriviaQA** [37] and **PopQA** [11]. As shown in Table A.11, our method generalizes effectively to text-only tasks, achieving the highest AUROC and AUARC scores on both datasets.

Table A.11: SGPU consistently improves AUROC and AUARC while reducing ECE. Comparison across QA datasets, metrics, and LLM architectures. **Bold** values indicate best performance.

Dataset	Method	Llama-3.1-8B		
		AUROC	AUARC	ECE
Trivia-QA	SE	0.821	0.836	0.255
	DSE	0.828	0.838	0.281
	Regular	0.752	0.819	0.060
	KLE-Heat	0.844	0.847	0.105
	KLE-Matern	0.860	0.851	0.078
	UMPIRE	0.839	0.821	0.310
	Cos Eigenscore	0.833	0.786	0.080
	Cov Eigenscore	0.724	0.740	0.240
	SGPU (ours)	0.870	0.867	0.015
Pop-QA	SE	0.739	0.618	0.130
	DSE	0.734	0.623	0.159
	Regular	0.678	0.646	0.334
	ManualHeat	0.664	0.646	0.455
	ManualMatern	0.705	0.626	0.656
	UMPIRE	0.871	0.570	0.050
	Cos Eigenscore	0.853	0.846	0.132
	Cov Eigenscore	0.784	0.753	0.114
	SGPU (ours)	0.908	0.902	0.037

F.2 Transferability

As discussed in Section 5, an interesting aspect is that the trained SGPU generalizes well across different LVLM architectures in plug-and-play manner without any adjustment or fine-tuning. For VQA tasks, Table 3 shows substantial cross-model transfer, where a model trained using outputs from one LVLM can be applied effectively to another with only a small loss in performance. This effect is even more pronounced in image classification. As illustrated in Table A.12, we observe almost no degradation in AUROC and AUARC when the training and test data are generated from different models. These results suggest that the distribution of eigenvalues is not strongly dependent on any specific LVLM models, and that our method exhibits robust cross-model generalization.

F.3 Complete Results

Table A.13 presents a summary of all results discussed in Section 5.

Table A.12: SGPU generalizes well across different LVLM architectures. Each row shows the SGPU trained on the outputs of one LVLM and tested on another for the **CIFAR** dataset. **Bold** values indicate best performance.

Trained GP	Tested on	AUROC	AUARC	ECE
Qwen2.5-VL-7B	Qwen2.5-VL-7B	0.887	0.885	0.007
Qwen2.5-VL-3B	Qwen2.5-VL-7B	0.851	0.854	0.091
llava-mistral-7b	Qwen2.5-VL-7B	0.885	0.876	0.078
idefics2-8b	Qwen2.5-VL-7B	0.882	0.866	0.055
Qwen2.5-VL-3B	Qwen2.5-VL-3B	0.830	0.810	0.010
Qwen2.5-VL-7B	Qwen2.5-VL-3B	0.821	0.771	0.916
llava-mistral-7b	Qwen2.5-VL-3B	0.822	0.792	0.085
idefics2-8b	Qwen2.5-VL-3B	0.763	0.676	0.015
idefics2-8b	idefics2-8b	0.867	0.892	0.013
Qwen2.5-VL-3B	idefics2-8b	0.799	0.863	0.124
Qwen2.5-VL-7B	idefics2-8b	0.862	0.888	0.069
llava-mistral-7b	idefics2-8b	0.862	0.883	0.109
llava-mistral-7b	llava-mistral-7b	0.945	0.925	0.011
Qwen2.5-VL-3B	llava-mistral-7b	0.923	0.917	0.133
Qwen2.5-VL-7B	llava-mistral-7b	0.944	0.919	0.084
idefics2-8b	llava-mistral-7b	0.941	0.802	0.124

Table A.13: SGPU consistently improves AUROC and AUARC while reducing ECE. Comparison across vision datasets, metrics, and LVLM backbones. **Bold** values indicates best performance, underline values indicate the second-best.

Dataset	Method	Qwen2.5-VL-3B			Qwen2.5-VL-7B			idefics2-8b			llava-mistral-7b			Mean		
		AUROC	AUARC	ECE	AUROC	AUARC	ECE	AUROC	AUARC	ECE	AUROC	AUARC	ECE	AUROC	AUARC	ECE
CIFAR10	SE	0.741	0.712	0.066	0.808	0.832	0.102	0.841	<u>0.876</u>	0.105	0.904	<u>0.909</u>	0.026	0.823	0.832	0.075
	DSE	0.750	0.716	0.101	0.802	0.829	0.062	0.837	0.874	<u>0.096</u>	0.903	<u>0.909</u>	0.022	0.818	0.832	0.070
	PE	0.732	0.700	0.110	0.782	0.814	0.072	0.836	<u>0.876</u>	0.099	0.898	0.907	0.043	0.812	0.824	0.081
	KLE-Heat	<u>0.796</u>	<u>0.742</u>	0.069	0.806	0.832	0.089	0.823	0.873	0.120	0.882	0.905	0.058	0.827	<u>0.838</u>	0.084
	KLE-Matern	0.787	0.736	0.096	0.809	<u>0.832</u>	<u>0.056</u>	0.832	0.873	0.098	0.899	0.908	<u>0.015</u>	0.832	0.837	<u>0.066</u>
	UMPIRE	0.724	0.629	0.097	0.793	0.798	0.187	0.856	0.873	0.144	<u>0.929</u>	0.901	0.060	0.826	0.800	0.122
	Cos Eigenscore	0.744	0.559	0.190	<u>0.840</u>	0.780	0.100	0.871	0.867	0.123	0.907	0.874	0.093	<u>0.841</u>	0.770	0.127
	Cov Eigenscore	0.660	0.510	<u>0.058</u>	0.819	0.778	0.209	0.841	0.863	0.430	0.826	0.849	0.275	0.787	0.750	0.243
	SGPU (ours)	0.830	0.810	0.010	0.887	0.885	0.007	<u>0.867</u>	0.892	0.013	0.945	0.925	0.011	0.882	0.878	0.013
	SE	0.778	0.811	0.181	0.739	0.754	0.387	0.734	0.787	0.156	<u>0.800</u>	<u>0.815</u>	0.205	0.763	0.792	0.232
OKVQA	DSE	0.750	0.800	<u>0.075</u>	0.740	0.750	0.398	0.732	0.785	0.146	0.799	<u>0.815</u>	0.234	0.753	0.787	0.213
	PE	0.747	0.800	<u>0.075</u>	0.705	0.738	0.164	0.726	0.787	0.161	0.769	0.806	0.037	0.737	0.783	0.109
	KLE-Heat	0.748	0.803	0.112	0.722	0.749	0.243	0.736	0.787	0.176	0.720	0.789	0.233	0.731	0.782	0.191
	KLE-Matern	<u>0.788</u>	<u>0.818</u>	0.082	<u>0.740</u>	0.756	<u>0.038</u>	0.739	0.788	0.144	0.782	0.813	<u>0.037</u>	0.762	<u>0.794</u>	<u>0.075</u>
	UMPIRE	0.773	0.768	0.112	0.758	<u>0.763</u>	0.485	0.752	0.790	0.187	0.796	0.787	0.161	0.768	0.777	0.236
	Cos Eigenscore	0.756	0.713	0.103	0.733	0.709	0.105	0.767	0.784	0.192	0.744	0.747	0.090	0.750	0.738	0.123
	Cov Eigenscore	0.640	0.644	0.406	0.707	0.696	0.133	0.715	0.768	0.166	0.677	0.713	0.416	0.685	0.705	0.280
	SGPU (ours)	0.840	0.840	0.020	0.730	0.770	0.020	<u>0.753</u>	0.798	0.150	0.829	0.832	0.022	0.788	0.810	0.039
	SE	0.681	0.636	0.220	0.703	0.663	0.346	0.734	0.648	0.318	0.647	0.537	0.246	0.691	0.621	0.282
	DSE	0.688	0.638	0.238	0.703	0.658	0.359	0.732	0.643	0.312	0.641	0.457	0.259	0.691	0.599	0.292
VQARAD	PE	<u>0.723</u>	0.662	0.054	0.692	0.649	<u>0.109</u>	0.731	0.650	0.132	0.672	0.534	<u>0.096</u>	0.705	0.620	0.132
	KLE-Heat	0.747	0.652	0.141	0.677	0.652	0.141	0.727	0.643	0.325	0.496	0.612	0.314	0.644	<u>0.640</u>	0.230
	KLE-Matern	0.720	<u>0.665</u>	0.120	<u>0.720</u>	<u>0.665</u>	0.120	0.729	0.643	0.296	0.567	0.564	0.100	0.684	0.634	<u>0.159</u>
	UMPIRE	0.688	0.597	0.184	0.704	0.617	0.321	0.768	0.654	0.381	<u>0.734</u>	0.621	0.152	<u>0.724</u>	0.622	0.260
	Cos Eigenscore	0.631	0.502	0.176	0.710	0.619	0.194	0.792	0.665	0.193	0.674	0.687	0.240	0.702	0.618	0.201
	Cov Eigenscore	0.654	0.513	0.202	0.684	0.604	0.329	0.771	0.650	0.108	0.672	0.696	0.296	0.695	0.616	0.234
	SGPU (ours)	0.770	0.770	0.060	0.780	0.710	0.060	<u>0.750</u>	<u>0.664</u>	0.278	0.758	0.815	0.045	0.752	0.740	0.108
	SE	0.688	0.742	0.098	0.657	0.669	0.147	0.630	0.574	0.303	0.716	0.647	0.050	0.673	0.658	0.149
	DSE	0.687	0.741	0.106	0.664	0.673	0.168	0.632	0.572	0.286	0.720	0.649	<u>0.046</u>	0.676	0.659	0.152
	PE	0.693	0.745	0.030	0.655	0.666	0.118	0.630	0.575	0.324	0.711	0.643	0.200	0.672	0.657	0.168
ADVQA	KLE-Heat	<u>0.724</u>	0.760	0.078	0.715	0.702	0.171	0.615	0.565	0.285	0.700	0.640	0.137	0.688	0.667	0.168
	KLE-Matern	<u>0.724</u>	0.760	0.097	0.718	0.701	<u>0.044</u>	0.628	0.571	0.254	0.730	0.656	0.026	<u>0.700</u>	<u>0.672</u>	<u>0.105</u>
	UMPIRE	0.627	0.659	0.187	0.641	0.631	0.163	<u>0.665</u>	0.583	0.378	<u>0.735</u>	0.613	<u>0.188</u>	0.667	0.621	0.229
	Cos Eigenscore	0.573	0.577	0.080	0.556	0.521	0.130	0.683	0.576	0.380	0.726	<u>0.675</u>	0.212	0.635	0.587	0.201
	Cov Eigenscore	0.575	0.583	0.355	0.613	0.561	0.218	0.637	0.550	0.200	0.656	0.629	0.524	0.620	0.581	0.324
	SGPU (ours)	0.740	0.750	0.030	<u>0.690</u>	<u>0.690</u>	0.030	<u>0.655</u>	<u>0.623</u>	0.057	0.757	0.740	0.049	0.711	0.701	0.040
	SE	0.818	<u>0.607</u>	0.186	0.790	0.649	0.234	0.741	0.702	0.204	0.790	0.671	0.254	0.785	0.670	0.219
	DSE	0.824	0.598	0.214	<u>0.792</u>	0.640	0.249	0.742	0.700	<u>0.183</u>	<u>0.791</u>	<u>0.666</u>	0.276	0.787	0.651	0.230
	PE	0.783	0.586	0.126	0.778	0.639	0.056	0.741	0.706	0.218	0.755	0.650	<u>0.046</u>	0.764	0.645	<u>0.112</u>
	KLE-Heat	0.746	0.566	0.261	0.767	0.635	0.236	0.743	0.701	0.225	0.726	0.632	0.218	0.746	0.634	0.235
VIZWIZ	KLE-Matern	0.785	0.592	0.260	0.763	<u>0.652</u>	<u>0.055</u>	0.748	0.703	0.187	0.764	0.657	<u>0.046</u>	0.765	0.651	0.137
	UMPIRE	<u>0.848</u>	0.522	0.120	0.776	0.607	0.214	0.780	<u>0.710</u>	0.244	0.789	0.627	0.182	<u>0.798</u>	0.616	0.193
	Cos Eigenscore	0.790	0.501	0.116	0.763	0.571	0.068	0.794	0.701	0.340	0.760	0.580	0.100	0.777	0.588	0.156
	Cov Eigenscore	0.686	0.572	<u>0.048</u>	0.701	0.533	0.098	0.757	0.685	0.396	0.701	0.546	0.092	0.711	0.588	0.156
	SGPU (ours)	0.881	0.872	0.032	0.818	0.808	0.021	<u>0.783</u>	0.777	0.019	0.828	0.814	0.037	0.828	0.819	0.027
	SE	0.681	0.593	0.239	0.583	0.573	0.250	0.510	0.733	0.414	0.643	0.570	0.333	0.604	0.617	0.309
	DSE	0.684	<u>0.594</u>	0.188	0.574	0.570	0.224	0.521	0.721	0.433	0.644	0.573	0.296	0.605	0.614	0.285
	PE	0.677	0.598	0.129	0.663	<u>0.647</u>	<u>0.125</u>	0.546	0.750	0.442	0.610	0.569	0.343	0.624	<u>0.641</u>	0.260
	KLE-Heat	0.667	0.584	0.324	0.600	0.581	0.359	0.486	0.714	0.364	0.602	0.585	0.439	0.589	0.616	0.372
	KLE-Matern	0.677	0.589	0.216	0.595	0.578	0.270	0.484	0.714	0.403	0.605	0.582	0.313	0.590	0.616	0.301
Imagette	UMPIRE	0.672	0.534	0.250	0.588	0.532	0.299	0.465	0.755	0.375	0.711	0.578	0.427	0.609	0.600	0.338
	Cos Eigenscore	0.700	0.504	0.274	0.679	0.550	0.352	0.507	0.764	0.371	0.612	0.598	0.345	0.625	0.604	0.336
	Cov Eigenscore	0.704	0.501	0.104	0.778	0.614	0.073	0.872	0.551	0.075	<u>0.721</u>	0.538	0.123	<u>0.769</u>	0.576	<u>0.094</u>
	SGPU (ours)	0.800	0.790	0.161	<u>0.740</u>	0.740	<u>0.157</u>	<u>0.669</u>	<u>0.761</u>	<u>0.101</u>	0.785	0.805	0.068	<u>0.749</u>	0.774	<u>0.121</u>
	SE	0.681	0.593	0.239	0.583	0.573	0.250	0.510	0.733	0.414	0.643	0.570	0.333	0.604	0.617	0.309